NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF ENGINEERING IN FOREIGN LANGUAGES
COMPUTERS AND INFORMATION TECHNOLOGY - INFORMATION ENGINEERING

DIPLOMA PROJECT

Project coordinator
Prof. Andrei VASILĂȚEANU

Student:
Bogdan-Laurențiu ȘTEFĂNESCU

Bucharest
2025

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF ENGINEERING IN FOREIGN LANGUAGES
COMPUTERS AND INFORMATION TECHNOLOGY -
INFORMATION ENGINEERING

Decentralized Identity Management for Web Applications Using Blockchain

Project coordinator
Prof. Andrei VASILĂȚEANU

Student:
Bogdan-Laurențiu ȘTEFĂNESCU

Bucharest
2025

Universitatea Națională de Știință și Tehnologie
POLITEHNICA București
Facultatea de Inginerie în Limbi Străine

National University of Science and Technology
POLITEHNICA Bucharest
Faculty of Engineering in Foreign Languages

Comisie pentru specializare
Specialization comission

Decan
Dean
Prof. dr. DASCĂLU Maria-Iuliana

TEMĂ PROIECT DE DIPLOMĂ
DIPLOMA PROJECT THEME

Subsemnatul, voi pregăti lucrarea de diplomă cu titlul:

Managementul descentralizat al identității pentru aplicații web folosind Blockchain

domeniul Calculatoare și tehnologia informației
specializarea Ingineria informației
pentru sesiunea 2025 Iulie .

I, the undersigned, will prepare the diploma thesis with the title:

Decentralized Identity Management for Web Applications Using Blockchain

field Computers and information technology
specialization Information engineering
for the 2025 July session.

Profesor îndrumător
Project coordinator

Prof. Dr. Ing. VASILĂȚEANU Andrei

Student
Student

ȘTEFĂNESCU Bogdan-Laurențiu

Academic Honesty Statement

I, **Bogdan-Laurențiu ȘTEFĂNESCU**, hereby declare that the work with the title "**Decentralized Identity Management for Web Applications Using Blockchain"**, to be openly defended in front of the diploma theses examination commission at the Faculty of Engineering in Foreign Languages, National University of Science and Technology "Politehnica" of Bucharest, as partial requirement for obtaining the title of Engineer is the result of my own work, based on my work.

The thesis, simulations, experiments and measurements that are presented are made entirely by me under the guidance of the scientific adviser, without the implication of persons that are not cited by name and contribution in the Acknowledgements part.

The thesis has never been presented to a higher education institution or research board in the country or abroad.

All the information used, including the Internet, is obtained from sources that were cited and indicated in the notes and in the bibliography, according to ethical standards. I understand that plagiarism is an offense and is punishable under law.

The results from the simulations, experiments and measurements are genuine. I understand that the falsification of data and results constitutes fraud and is punished according to regulations.

Bogdan-Laurențiu ȘTEFĂNESCU

Contents

1. Introduction 1
2. State of the Art 3
2.1. LDAP and Active Directory 3
2.2. SAML, OAuth and OIDC 5
2.2.1. OAuth 2.0: Authorization Protocol 5
2.2.2. OpenID Connect (OIDC): Identity Protocol 6
2.2.3. Security Assertion Markup Language (SAML) 7
2.2.4. Comparison 8
2.3. eIDAS 9
2.3.1. eIDAS v1 (EU Regulation 910/2014) 9
2.3.2. eIDAS v2 (EU Regulation 1183/2024) 9
2.3.3. Challenges 10
2.3.4. Implementations 11
2.4. FIDO, WebAuthn, and Passkeys 13
2.4.1. FIDO1 13
2.4.2. FIDO2 and WebAuthn 14
2.4.3. Passkeys 15
2.4.4. Challenges and Issues 16
2.4.5. Prospects 18
2.5. Decentralised Identities 20
3. Requirements Analysis 22
3.1. Actors and Business Agents 22
3.2. Business Rules 23
3.3. Use Cases 24
3.4. Non-Functional Requirements 28
4. System Design 30
5. Implementation 33
5.1. Identity Gateway 33
5.2. Country Nodes – Application Registries 36
5.2.1. Application Registry 37
5.2.2. Identity Node 42
5.3. Identity Providers 48
5.3.1. Banana Country 49
5.3.2. Orange Country 55
6. Future Work 60
6.1. Decentralized Identity Resolution and Duplication Prevention 60
6.2. Country nodes admin panel 62
6.3. User migration flow 63
7. Conclusions 65

List of Tables and Figures

Figure 2.1: Experimental eIDAS node extended with support for new attributes [10] 11
Figure 2.2: Ethereum login message requests [21] 21
Figure 3.1: Use Case Diagram 28
Figure 4.1: Login flow (simplified) 31
Figure 5.1: Ghiseul.ro eIDAS Login Page [24] 34
Figure 5.2: Gateway discovery sequence diagram 35
Figure 5.3: Login page MetaMask pre-fill consent form 40
Figure 5.4: User Authorization Flow (Application Registry) 41
Figure 5.4: Same country authentication sequence diagram. 43
Figure 5.5: Different countries authentication sequence diagram 45
Figure 5.6: Banana IDP Sequence Diagram 50
Figure 5.7: Banana Login Interface with Passkey Prompt 50
Figure 5.8: Banana IDP Passkey Authentication Flow 51
Figure 5.9: Banana IDP Account Page 52
Figure 5.10: Banana IDP Audit Log Page 52
Figure 5.11: Orange IDP Login Page with Signature Request 56
Figure 5.12: Orange IDP Registration Form 57
Figure 5.13: Orange IDP Identity Verification Page 58
Figure 6.1: Authentication Flow with Smart Contract Discovery 60

Table 2.1: OAuth 2.0 Authorization Flows [6] 5
Table 3.1: Business Rules Table 23
Table 3.2: Use Case 1 – Register New Application 24
Table 3.3: Use Case 2 - Modify Application Scopes 25
Table 3.4: Use Case 3 - Perform Federated Login 25
Table 3.5: Use Case 4 - Register New User 26
Table 3.6: Use Case 5 - Discover User Home Node 26
Table 3.7: Use Case 6 – Application Scope Validation 27
Table 3.8: Use Case 7 - Node Data Exchange 27
Table 5.1: Custom HTTP Headers Used 34
Table 5.2: ApplicationScope Class Table 37
Table 5.3: Application Class Table 37
Table 5.4: Scopes List 38
Table 5.5: Country Node - Application Registry Endpoint List 39
Table 5.6: Country Node - Identity Node Endpoint List 42
Table 5.7: Tokens Issued During OAuth 2.0 Authorization Code Exchange 47
Table 5.8: Banana IDP Database Structure 49
Table 5.9: Banana IDP Audit Log Events 53
Table 5.10: Banana IDP Endpoint List 54
Table 5.11: Orange IDP Endpoint List 56
Table 5.12: Orange IDP Collections List 58
Table 5.13: Orange IDP Smart Contract Endpoints 59

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
Dapp	Decentralized Application
DG	Data Group
DID	Decentralized Identifier
eID	Electronic Identification
eIDAS	Electronic Identification, Authentication and Trust Services
EU	European Union
FIDO	Fast Identity Online
GDPR	General Data Protection Regulation
IdP	Identity Provider
JSON	JavaScript Object Notation
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
MFA	Multi-Factor Authentication
NFC	Near Field Communication
OAuth	Open Authorization
OIDC	OpenID Connect
RSA	Rivest-Shamir-Adleman Algorithm
REST	Representational State Transfer
SAML	Security Assertion Markup Language
SSO	Single Sign-On
TPM	Trusted Platform Module
U2F	Universal 2nd factor
UAF	Universal Authentication Framework
W3C	World Wide Web Consortium

1. Introduction

IN today's interconnected digital landscape, identity management remains a cornerstone for secure and seamless user experiences. Traditional identity systems, often based on centralized directory services, suffer from limitations including fragmentation, lack of user control, and significant privacy concerns. These models typically rely on monolithic authentication providers or Single Sign-On (SSO) frameworks within narrowly scoped domains, forcing users to repeatedly register and verify themselves across different jurisdictions and service providers.

Regulatory frameworks such as eIDAS have sought to improve cross-border digital identity verification, yet challenges remain, particularly around the user experience, where manual intervention or static country-based mappings are required to determine an identity's origin. Decentralized identity systems promise to overcome these barriers by leveraging cryptographic proofs and distributed trust models. However, deploying a truly interoperable, privacy-preserving decentralized identity infrastructure that integrates seamlessly with real-world application ecosystems remains an ongoing challenge.

This project introduces a novel decentralized login system that addresses these shortcomings by federating identity across autonomous national "nodes", each responsible for managing local application registries and linking to country-specific identity providers. Unlike legacy systems, this architecture dynamically determines the user's origin, regardless of the application or country of access, without imposing a centralized authority. Each country's node operates as an independent unit within a broader federated network, holding essential identity information for its jurisdiction while collaborating in real time through a custom authentication flow that respects both user privacy and data minimization principles.

The project envisions a flexible and user-centric identity ecosystem, where applications, whether domestic or foreign, can verify identities in a trusted and decentralized manner. This dynamic federation model supports both blockchain-based and traditional identity validation, reflecting the growing demand for hybrid architectures that merge emerging decentralized paradigms with established identity frameworks. By building on these foundations, the system aims to demonstrate a practical, scalable approach to decentralized identity management suitable for modern, globally interconnected web applications.

The design of this system is also deeply rooted in personal development experiences. Throughout the course of working on various personal and academic projects, I found that nearly every application required building a login system from scratch. This repeated effort quickly became inefficient and error-prone, especially as the complexity of projects increased. To streamline this, I began developing a custom OAuth server, initially intended to provide a common login backend for my personal tools. As more projects adopted it, I expanded it to include structured identity data in an OpenID Connect–like format, allowing applications to not only authenticate users but also retrieve consistent profile information. This internal identity platform became a foundation for both personal and university-related applications, ultimately planting the seeds for a more general, federated system.

A more formative moment came during the development of an IoT-based identity solution for student attendance tracking. In this project, I designed a mobile application capable of scanning electronic passports and national ID cards via NFC (Near Field Communication), following the same cryptographic protocols used in border security. The app implemented Basic Access Control (BAC) to establish a secure communication channel with the chip, requiring a session key derived from the machine-readable zone (MRZ) printed on the document. Once access was granted, the system read data from the chip's Logical Data Structure: DG1, which contains personal information such as the name, birth date, and document number, and DG5, which includes the printed photograph from the document for visual comparison.

To verify the authenticity of the scanned identity documents, the application implemented Passive Authentication, a standard cryptographic procedure used in electronic travel documents. In this process, the application first calculated the hash of the data contained in DG1 and compared it against the signed hash values stored in the document's Security Object. These signatures are issued by the document's originating authority and are digitally signed using a national root certificate. The application validated these signatures using public keys from a trusted source; specifically, the International Civil Aviation Organization Public Key Directory, which maintains a curated list of legitimate country signing certificate authorities. This validation process ensured both the integrity of the document data and the authenticity of its issuer, confirming that the chip had not been altered and that the document was issued by a recognized and trusted state authority. Because all of these checks can be performed locally without requiring an internet connection, the solution provided a high-assurance, offline-capable identity verification mechanism, fully aligned with internationally recognized standards.

To close the attendance loop, I implemented a direct NFC-to-NFC communication channel between the student's phone and the teacher's device. After securely reading and validating the identity document, the student's phone would transmit the identity proof directly to the teacher's terminal, enabling a secure and privacy-respecting tap-to-check-in experience. The system avoided reliance on centralized infrastructure for runtime validation, instead leveraging standards-compliant cryptographic assurance and local device trust, aligning closely with the decentralised identity principles later applied in this thesis.

2. State of the Art

THE evolution of identity management has been shaped by a steady progression from isolated, application-specific user accounts to federated and, more recently, decentralized models. Early systems relied on centralized directories such as LDAP [1] to store and authenticate user credentials within organizational boundaries. Over time, the need for seamless cross-application access and interoperability gave rise to federated identity management and Single Sign-On (SSO) solutions, including widely adopted protocols like SAML and OpenID Connect [2]. While these frameworks have addressed some challenges of isolated identity silos, they remain dependent on trusted third-party providers and introduce new privacy and security considerations.

The increasing demand for user-centric privacy, portability, and trust minimization has driven the exploration of decentralized identity frameworks. Blockchain-based systems, leveraging cryptographic proofs and distributed ledgers, promise to overcome the limitations of traditional identity systems. In parallel, government-driven initiatives such as eIDAS [3] [4] have sought to standardize cross-border digital identity frameworks, though often with a more centralized architecture that contrasts with emerging self-sovereign identity models.

This section examines the key technologies, protocols, and regulatory frameworks that form the foundation of modern identity management. It explores the historical context and the current state-of-the-art, highlighting their respective strengths and limitations, and provides essential background for understanding how decentralized identity systems can bridge the gap between user autonomy and secure authentication in the digital age.

2.1. LDAP and Active Directory

The Lightweight Directory Access Protocol (LDAP) is a foundational protocol for managing identity information within organizations. Originally developed as a simplified alternative to the X.500 Directory Access Protocol, LDAP provides a standardized way to query and manage directory entries in a hierarchical structure. Each entry is uniquely identified by a Distinguished Name (DN) and consists of attribute-value pairs that describe user identities, organizational units, and resources. [5]

LDAP became the backbone of centralized identity management in enterprise environments. It enables applications and services to authenticate users by checking credentials against a centralized directory. This reduces data redundancy and streamlines administration by consolidating user information in one place. LDAP's read-optimized, hierarchical structure supports fast lookups of identity attributes, making it suitable for managing large numbers of users and groups. However, LDAP is inherently centralized, creating a single point of failure and a dependency on trusted administrators. In modern web-based applications, integrating

LDAP directly can be cumbersome, as it relies on binary encodings and a complex protocol that does not align with typical REST-based API designs.

Active Directory (AD) is a common repository for information about objects that reside on the network, such as users and groups, computers and printers, and applications and files. [1] Developed by Microsoft, Active Directory is built upon LDAP and was extended to provide a comprehensive identity and access management system for Windows-based environments. Active Directory uses LDAP as the protocol for directory access, while integrating additional services such as Kerberos-based authentication, Group Policy management, and domain trust relationships. AD's directory service provides a unified interface for authentication and authorization within a Windows domain. Administrators can manage permissions, apply security policies, and define organizational structures directly in AD, streamlining identity management for complex IT environments.

Both LDAP and AD have played crucial roles in the evolution of digital identity systems. They offer:

- Centralized control over identity data, ensuring consistency and reducing administrative burden.
- Scalability for large organizations with many users and resources.
- Integration with enterprise applications and networks, especially within Windows-based infrastructures.

Yet, they also present limitations in modern, web-oriented ecosystems. LDAP and AD rely on a single administrative authority, leading to a single point of failure and trust dependency. They also lack portability and user control, which has driven the emergence of decentralized identity paradigms where users manage their own digital credentials and cryptographic keys.

In the context of decentralized identity, LDAP and AD highlight the contrast between traditional centralized identity directories and emerging self-sovereign identity (SSI) frameworks. While these directories remain integral to many enterprise environments, they underscore the need for new approaches that combine strong security, interoperability, and user autonomy.

2.2. SAML, OAuth and OIDC

In modern digital identity systems, three core protocols enable secure authentication and authorization across organizational boundaries: SAML, OAuth 2.0, and OpenID Connect (OIDC). Each protocol addresses specific use cases and security considerations, offering a variety of approaches for Single Sign-On (SSO) and delegated authorization. [2]

2.2.1. OAuth 2.0: Authorization Protocol

OAuth 2.0 is primarily an authorization protocol that allows third-party applications to access user resources without exposing the user's credentials. [6] It provides a mechanism for delegated access control by issuing access tokens that define what resources a client can access. Typically, these access tokens do not themselves contain user information, focusing solely on authorization.

OAuth 2.0 includes multiple grant types (flows) to support diverse security scenarios:

Table 2.1: OAuth 2.0 Authorization Flows [6]

Grant Type	Use Case	Security Considerations
Authorization Code	Web & Mobile Apps (Secure)	Uses a redirect-based flow; PKCE is required for mobile apps to prevent interception.
Implicit	Single Page Applications (SPA)	Returns tokens in URL fragments, exposing them to potential leakage.
Client Credentials	API communication	No user authentication: grants service-level authorization.
Resource Owner Password	Legacy apps (not recommended)	Directly uses user credentials, which is discouraged for modern security.

Token Structure and Security

- Access Tokens: Short-lived tokens for resource access, typically formatted as JSON Web Tokens (JWTs).
- Refresh Tokens: Longer-lived tokens to obtain new access tokens without re-authentication.
- Scopes: Define the specific resources or actions the client can access.
- Proof Key for Code Exchange (PKCE): A security enhancement to prevent code interception in mobile and public clients.

2.2.2. OpenID Connect (OIDC): Identity Protocol

OpenID Connect (OIDC) is an identity layer built on top of the OAuth 2.0 framework, enabling secure user authentication in addition to delegated authorization. Developed by the OpenID Foundation in 2014, OIDC addresses a key limitation of OAuth 2.0: OAuth provides authorization but not authentication. OIDC adds this missing layer, creating a unified framework for secure user login and data access.

OIDC uses the same four roles as OAuth 2.0, Resource Owner, Client, Authorization Server, and Resource Server, but introduces an additional standardized endpoint: the Userinfo Endpoint. This endpoint enables clients to retrieve profile information about the authenticated user.

Key OIDC components:

- ID Token: A JWT containing information about the user (claims), such as name, email, and a unique identifier (sub claim).
- Discovery Endpoint: Clients dynamically obtain configuration and key information from a standardized /.well-known/openid-configuration endpoint, supporting easier and more secure integration.

OIDC Authentication Flow

The OIDC flow builds directly on OAuth 2.0's Authorization Code Flow:

- User initiates login: The client redirects the user to the IdP's authorization endpoint, requesting authentication and desired scopes.
- User authenticates at the IdP: Via password, MFA, biometrics, or other supported methods.
- Authorization Code issued: Upon successful login, the IdP returns an authorization code to the client.
- Token exchange: The client exchanges the authorization code for an Access Token (for APIs) and an ID Token (for user information).
- ID Token validation: The client verifies the ID Token's signature, issuer, audience, and expiration to ensure authenticity.

2.2.3. Security Assertion Markup Language (SAML)

Security Assertion Markup Language (SAML) is an XML-based open standard for federated identity management and Single Sign-On (SSO). Standardized by the Organization for the Advancement of Structured Information Standards in 2005, SAML 2.0 enables secure and seamless access to multiple web applications by exchanging authentication and authorization data between trusted parties. [2]

SAML was created to solve the problem of "login fatigue" and duplicated credentials in enterprise environments. Before SAML, applications operated in silos, requiring users to maintain separate usernames and passwords for each service. SAML introduced a way for an organization to centralize authentication in an Identity Provider (IdP) and share trust with multiple Service Providers (SPs). This approach laid the foundation for secure cross-domain SSO, especially in large organizations and academic federations.

A typical SAML architecture includes three main actors:

- User (Principal): The individual seeking access to a service.
- Identity Provider (IdP): The authoritative source that verifies the user's identity (e.g., a corporate identity management system or a university's central login server).
- Service Provider (SP): The application or resource that the user wants to access (e.g., cloud-based software, internal enterprise applications, academic research services).

The SAML flow consists of several steps:

1. User attempts access: The user tries to access a service protected by SAML.
2. SP redirects to IdP: The SP issues a SAML AuthnRequest and redirects the user's browser to the IdP.
3. User authentication at IdP: The IdP authenticates the user (via password, MFA, or other mechanisms).
4. SAML Assertion creation: The IdP generates a digitally signed SAML Assertion containing the user's authentication status and optional attributes (e.g., roles, group memberships).
5. Assertion delivery: The assertion is transmitted via the user's browser back to the SP, typically using HTTP-POST binding.
6. SP verification: The SP validates the assertion's digital signature using the IdP's public key, ensuring authenticity and integrity.
7. Access granted: If the assertion is valid and the user has the appropriate permissions, the SP grants access.

2.2.4. Comparison

OAuth 2.0 is primarily an authorization framework designed to grant limited access to resources without sharing user credentials. [6] It uses JSON Web Tokens (JWT) as access tokens and typically issues short-lived tokens that can be refreshed without user intervention. While OAuth 2.0 allows for flexible and lightweight delegated access, it does not directly handle user authentication. Security risks associated with OAuth 2.0 include token theft, scope abuse, and man-in-the-middle (MitM) attacks. Mitigation strategies such as PKCE (Proof Key for Code Exchange), the evolving OAuth 2.1 drafts, and mutual TLS (mTLS) are critical for secure deployments. Although OAuth 2.0 does not have built-in support for Multi-Factor Authentication (MFA), it can integrate external IdPs that provide MFA.

OpenID Connect (OIDC) builds directly on OAuth 2.0, adding an identity layer that enables authentication and identity verification. OIDC uses ID Tokens: JWTs containing claims about the user's identity, alongside access tokens for resource authorization. This makes OIDC a more comprehensive protocol for applications requiring both authorization and secure authentication. OIDC addresses some of OAuth 2.0's security challenges through nonce parameters and claims-based access control, protecting against token replay and phishing attacks. OIDC also integrates easily with MFA provided by the IdP, improving security further. In practice, OIDC's use of lightweight JSON-based tokens and RESTful APIs makes it ideal for cloud-native and mobile-first applications, with moderate processing overhead and highly scalable deployments. SAML, on the other hand, is an XML-based standard for federated authentication and Single Sign-On (SSO). Unlike OAuth 2.0 and OIDC, which rely on JWT and JSON-based tokens, SAML uses verbose XML assertions. These assertions can include rich user attributes and are signed and encrypted to ensure authenticity and confidentiality. SAML is heavily adopted in enterprise environments, academic federations, and government applications. While it excels in environments requiring rich attribute sharing and strong cross-domain authentication, its stateful architecture and XML processing introduce higher processing overhead and make it less efficient in API-based and cloud-native scenarios. Security concerns in SAML implementations include XML signature wrapping attacks and assertion replay. SAML addresses these risks with XML signatures and one-time assertions.

In terms of performance and scalability, OAuth 2.0 and OIDC stand out due to their compact, stateless JSON token format, enabling low overhead and seamless scalability in microservices and modern web applications. SAML's reliance on XML parsing and its stateful nature create heavier payloads and more complex processing, limiting its scalability in API-centric deployments.

Overall, the evolution from SAML to OAuth 2.0 and OIDC represents a shift from enterprise-focused, XML-based federated SSO to lightweight, developer-friendly, JSON-based authorization and authentication for modern web and cloud ecosystems. Despite its limitations, SAML remains a critical component in many established organizations, while OAuth 2.0 and OIDC have become the dominant standards in today's cloud-native identity management.

2.3. eIDAS

"The eIDAS regulation facilitates secure cross-border transactions by establishing a framework for digital identity and authentication. It aims to create confidence in electronic interactions and promote seamless digital services in the EU." [7]

2.3.1. eIDAS v1 (EU Regulation 910/2014)

The first version of eIDAS, formalized as Regulation (EU) No. 910/2014 [3], set out to create a harmonized framework for the recognition of electronic identification (eID) and trust services across EU Member States. It enabled the mutual recognition of notified national eID schemes and gave legal weight to electronic signatures, seals, timestamps, and delivery services. A technical architecture based on eIDAS Nodes, combining national connectors and proxies, ensured interoperability by exchanging authentication data using SAML 2.0 protocols. This approach laid the groundwork for secure cross-border access to public services, reflecting the EU's broader ambition to build a cohesive Digital Single Market.

Despite these achievements, eIDAS v1 relied exclusively on government-issued and controlled eID models. Users had little agency in managing their personal identity data, and the system's static, state-centric structure offered limited flexibility for sector-specific or user-managed scenarios. National differences in legal and technical implementation also led to inconsistencies in user experience and significant barriers for private-sector integration. For instance, service providers often lacked clear, uniform guidelines on how to implement and comply with eIDAS standards, leading to confusion and fragmented adoption.

2.3.2. eIDAS v2 (EU Regulation 1183/2024)

Recognizing the growing importance of user control and the emergence of decentralized identity solutions, eIDAS v2 (Regulation (EU) 2024/1183 [4]) reimagines the framework to better align with the principles of the digital era. Central to eIDAS v2 is the introduction of the European Digital Identity Wallet (EUDI Wallet), which every Member State must offer to citizens and legal entities. These wallets provide secure, user-controlled storage and selective sharing of verified identity data and credentials, such as diplomas or driving licenses. The regulation also broadens the scope of trust services, including qualified electronic ledgers that recognize the relevance of distributed ledger technology (DLT) for secure, tamper-evident records.

While eIDAS v2 maintains the legal trust anchors established by its predecessor (through the roles of qualified trust service providers and notified eID schemes), it also embraces the flexibility and privacy-preserving features of decentralized identity paradigms like Self-Sovereign Identity (SSI). Dynamic discovery mechanisms replace the static mappings of eIDAS v1, enabling more seamless cross-border identity verification and reducing friction for users navigating services outside their home country.

2.3.3. Challenges

The implementation of eIDAS, both in its original form and in the newly adopted v2 framework, has faced a complex array of challenges, rooted in legal, technical, and cultural differences across EU Member States. The comparative case study of Estonia and the Netherlands reveals how these challenges manifest in practice.

Estonia, known for its advanced eID ecosystem with solutions like Mobile-ID and Smart-ID, still faces limitations in ensuring consistent cross-border recognition, especially for legal entity representation and integration of sector-specific attributes. [8] Similarly, the Netherlands, with its well-established DigiD and eHerkenning frameworks, has been proactive in offering robust eID solutions for citizens and businesses, yet struggles to align national implementations with eIDAS standards, particularly in integrating emerging technologies such as biometrics.

Both countries highlight the broader issues facing eIDAS: the lack of a common EU identifier for individuals and legal entities, which impedes seamless cross-border transactions. Experts have also pointed out that interpretation of eIDAS provisions varies significantly, leading to different assurance levels and technical configurations across Member States. This diversity translates into fragmented user experiences; while eIDAS aims to create a uniform digital landscape, citizens and businesses often encounter inconsistent interfaces and different levels of service when interacting across borders. [8]

From a legal and governance perspective, the absence of a coherent framework for conformity assessments and supervision creates ambiguity. Member States differ in how they notify eID schemes, manage liability, and supervise trust service providers. These gaps leave service providers uncertain about their obligations and can slow adoption. [9]

Another notable challenge is the integration of decentralized identity principles, especially as eIDAS 2.0 introduces the European Digital Identity Wallet. While this wallet is meant to empower users and embrace Self-Sovereign Identity (SSI) features like selective disclosure, it simultaneously relies on legally mandated trust service providers to ensure accountability. Balancing these decentralized elements with the need for robust legal oversight has emerged as a central tension in eIDAS 2.0’s rollout.

Experts also emphasize the importance of user trust and experience. Language barriers, accessibility gaps, and variable quality of support services complicate usage, especially for citizens needing cross-border services. Addressing these challenges requires not only regulatory updates but also continuous collaboration among policymakers, service providers, and users to refine governance, enhance standardization, and improve the user experience.

2.3.4. Implementations

The implementation of eIDAS in real-world scenarios has highlighted both the system's transformative potential and its complex challenges. In the academic domain, two key projects showcase the integration of eIDAS in university and campus contexts: the Electronic Identification for Universities and Providing Login and Wi-Fi Access Services With the eIDAS Network initiatives.

The project "Electronic Identification for Universities [10]" explored how to extend eIDAS to support academic use cases, specifically for cross-border student mobility and academic record sharing. Recognizing that traditional eIDAS frameworks typically transport only a minimum set of personal data, the project focused on defining and supporting additional academic attributes essential for academic services, such as the student's current field of study, degree name, graduation year, and language proficiency.

To achieve this, the researchers developed an Attribute Provider (AP) Connector module. This module acted as an adapter, enabling the secure retrieval of these attributes from university information systems and integrating them into the eIDAS infrastructure. They implemented this architecture in a controlled experimental testbed, including Docker-based deployment of eIDAS nodes and attribute providers, and used it to build two eIDAS-enabled academic services:

- eRegistration: Designed for Erasmus student exchange programs, this service allowed universities to dynamically retrieve and integrate academic attributes into the student registration process. This reduced manual data entry, enhanced data accuracy, and streamlined cross-border academic collaboration.
- eLogin: Extended traditional university login systems by enabling national eID credentials to be used for access to campus services. It provided a fallback mechanism in cases of account loss, adding an extra layer of security and convenience for students and staff.

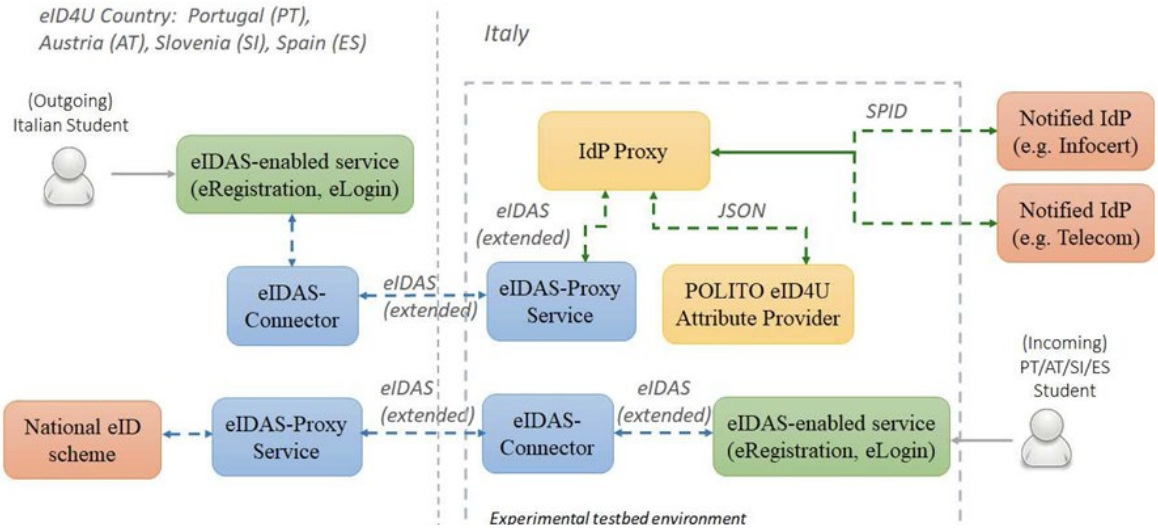


Figure 2.1: Experimental eIDAS node extended with support for new attributes [10]

The implementation experience highlighted the technical complexity of integrating sector-specific academic attributes into the eIDAS infrastructure, which originally focused only on government-issued eIDs and basic personal information. It also underscored the need for robust data mappings and adaptable user interfaces to ensure that these systems genuinely enhance the user experience.

Another example of eIDAS's flexibility is found in extending it to campus Wi-Fi authentication and federated login systems like eduroam [11]. This project [12] demonstrated how eIDAS can be leveraged to reduce fragmentation in identity management for students and researchers on the move. In traditional scenarios, universities and research institutions rely on separate, local login credentials for Wi-Fi access. The project explored how to bridge national eID schemes with existing Wi-Fi access protocols like RADIUS, allowing users to authenticate with their national eID credentials even when abroad. This integration required careful mapping of identity attributes between the eIDAS-provided information and the attributes needed for secure RADIUS-based authentication in eduroam.

By using eIDAS as a trusted authentication layer, the project aimed to provide seamless and secure access to Wi-Fi networks across Europe, particularly for traveling researchers and students participating in cross-border academic programs. It also addressed data minimization and privacy concerns, ensuring that only the necessary attributes were transferred during authentication.

Both projects underscore the broader challenges and opportunities of extending eIDAS beyond public-sector use. They reveal the technical hurdles of mapping eIDAS attributes to sector-specific protocols (like campus Wi-Fi or academic information systems), the legal and privacy concerns involved in handling sensitive academic data, and the organizational challenges of coordinating multiple stakeholders across borders. These practical implementations demonstrate that while eIDAS has the potential to enable a more secure and user-friendly digital identity ecosystem, it requires continuous refinement, sector-specific customization, and close collaboration between universities, trust service providers, and regulatory bodies. In this way, they lay the groundwork for a more inclusive and dynamic European digital identity framework, one that goes beyond e-government to truly support the diverse needs of students, researchers, and knowledge-sharing communities.

2.4. FIDO, WebAuthn, and Passkeys

The increasing sophistication of phishing attacks and the persistent vulnerabilities of passwords have driven the search for more secure, scalable, and user-friendly authentication mechanisms. The FIDO (Fast Identity Online) family of standards was developed to eliminate the reliance on passwords by leveraging public-key cryptography for secure user authentication. From its first iterations focused on physical security keys to today's passkey-enabled platforms, FIDO has progressively shaped the modern identity ecosystem.

This section explores the historical and technical progression from FIDO1 to FIDO2/WebAuthn, the rise of passkeys as a user-centric implementation of these standards, and the challenges and opportunities that accompany their adoption.

2.4.1. FIDO1

The first generation of FIDO standards, introduced by the FIDO Alliance in 2014, aimed to reduce dependence on passwords by leveraging asymmetric cryptography and hardware-based authentication. This early standard consisted of two primary protocols: Universal Second Factor (U2F) and Universal Authentication Framework (UAF) [13].

U2F was designed to work alongside existing password-based login systems, providing an additional authentication layer through a physical device, commonly a USB or NFC token. [13] After a user entered their password, they were prompted to interact with the second factor, which signed a server-provided challenge using a private key securely stored on the device. This approach prevented phishing, replay, and man-in-the-middle attacks, as the credential was both origin-bound and non-exportable. Companies like Google were early adopters, mandating U2F security keys for employee access as early as 2017.

UAF, in contrast, introduced a fully passwordless login flow. Users authenticated locally via biometrics (e.g., fingerprint or facial recognition) or a PIN, which unlocked a private key stored on their device. The private key then signed a challenge issued by the server, completing authentication without ever transmitting a shared secret. UAF was primarily aimed at mobile and consumer devices, reflecting early trends in biometric authentication.

While both protocols shared strong cryptographic foundations and significantly improved security over passwords, their practical deployment was limited. One reason was ecosystem fragmentation: U2F and UAF required specific client software or browser extensions, lacked native platform support, and had inconsistent integration pathways. Additionally, many users found hardware keys cumbersome or confusing to use, and registration processes were not standardized across services.

Another significant barrier to adoption was portability. Credentials in both U2F and UAF were typically bound to a single device. This meant that if a user lost or changed devices, they often had to re-register their credentials. For UAF, which relied heavily on platform-specific biometric systems, this also created interoperability problems between ecosystems (e.g., Android and iOS).

Nonetheless, FIDO1 laid the foundation for a modern rethinking of user authentication. It introduced key concepts that endure in later standards: local user verification, device-bound key pairs, and origin-specific challenges. Importantly, these limitations prompted the development of FIDO2, a more flexible and web-native successor designed to improve usability, interoperability, and scalability.

2.4.2. FIDO2 and WebAuthn

The limitations of FIDO1, particularly its reliance on external hardware tokens, inconsistent platform integration, and poor credential portability led to the development of FIDO2, a more robust and web-native solution designed to support both second factor and fully passwordless authentication scenarios.

FIDO2 is built on two key components: the Web Authentication (WebAuthn) API, standardized by the World Wide Web Consortium (W3C), and the Client to Authenticator Protocol (CTAP), defined by the FIDO Alliance. Together, these components enable modern web browsers and applications to interact with authentication hardware in a secure, standardized, and phishing-resistant manner. [14]

At the core of the FIDO2 protocol is a public–private key pair, generated by the client during registration with a web application. The private key is securely stored on a platform or external authenticator (such as a Trusted Platform Module (TPM), Secure Enclave, or security key) and is never exposed outside the device. The public key, alongside associated metadata, is registered with the relying party (i.e., the application server). To authenticate, the server sends a cryptographic challenge to the browser, which forwards it to the authenticator. After user verification (typically biometric or PIN), the device signs the challenge with the private key, and the signature is returned to the server for validation. [14]

This model offers significant security advantages. Because the key pair is origin-bound, credentials cannot be used across domains, eliminating phishing threats. Additionally, the lack of a shared secret means there’s nothing for attackers to steal from servers, mitigating mass credential breaches. Authentication is further protected by user presence and user verification checks, ensuring that only the legitimate user can authorize the action.

FIDO2 is also notable for being backwards-compatible with FIDO1's U2F protocol [13], allowing some legacy authenticators to operate within newer frameworks under certain constraints.

Unlike FIDO1, FIDO2 and WebAuthn gained broad support across platforms and browsers, including Chrome, Firefox, Safari, and Edge, and across operating systems such as Windows, Android, iOS, and macOS. WebAuthn has been integrated into major identity platforms like Microsoft Azure AD, Okta, and AWS Cognito, allowing enterprises and developers to implement passwordless login in real-world environments.

However, despite these advantages, real-world adoption of FIDO2 has been slower than expected. As highlighted by Lassak et al. (2024) [15], organizations face several deployment challenges:

- Many enterprise systems remain tightly coupled to legacy password flows, making integration costly or disruptive.
- Device-bound credentials limit usability across multiple devices, forcing users to re-enrol credentials on every device they use.
- Recovery processes are often poorly defined; if a device is lost and no backup credential exists, account access can be permanently lost.

Despite these issues, FIDO2 represents a significant leap forward in authentication architecture. It establishes a foundation on which more user-friendly models, such as passkeys, have been built. By enabling secure, standardized, passwordless login directly in the browser, FIDO2/WebAuthn helps shift the identity ecosystem away from shared secrets and toward cryptographically verifiable identity assertions that preserve user privacy and reduce reliance on vulnerable legacy systems.

2.4.3. Passkeys

While FIDO2 and WebAuthn addressed many of the shortcomings of earlier authentication systems, they introduced new usability barriers, particularly around device binding and credential portability. Users were required to re-register their credentials on each new device, which led to friction, abandonment, and increased help desk burden in enterprise environments. To overcome this, leading technology providers introduced a new concept: passkeys.

Passkeys are an evolution of FIDO2 credentials designed to enable secure, seamless, cross-device authentication without passwords. Technically, a passkey is a synchronized WebAuthn credential that retains all the core security properties of FIDO2 (origin binding, phishing

resistance, and non-reusability) but removes the requirement for manual re-enrolment across devices. Passkeys achieve this by allowing the private key component of the credential to be securely synced across the user's devices through end-to-end encrypted cloud storage.

Apple, Google, and Microsoft have all implemented support for passkeys within their respective ecosystems. For example, Apple's iCloud Keychain and Google's Password Manager now store passkeys alongside traditional credentials, but in a format that supports biometric unlocking, hardware isolation, and cryptographic signature operations without user-visible complexity. These credentials are securely stored using hardware-backed security modules (e.g., Apple's Secure Enclave or Android's StrongBox), and synchronized using encrypted, authenticated channels.

Passkeys not only address device portability but also improve cross-platform user experiences. For instance, a user can log in to a web service on a Windows laptop using a passkey stored on their iPhone, by scanning a QR code and confirming the login with Face ID. This is made possible by WebAuthn's cross-device authentication flow and increasingly standardized through proximity APIs like Bluetooth LE and cloud credential relaying. [14]

From a security perspective, passkeys preserve the phishing resistance and credential isolation offered by FIDO2. Each credential is still origin-bound, and private keys never leave the user's device unencrypted. Unlike traditional passwords or even OTP-based two-factor authentication, passkeys are not susceptible to credential reuse, shoulder surfing, or man-in-the-middle interception. They reflect a practical convergence of usability and security. They maintain the cryptographic assurances of FIDO2 while solving for the everyday challenges of real-world authentication: device loss, recovery, and login from new or shared environments. Importantly, they achieve this without requiring users to understand cryptography or maintain physical tokens.

Despite their benefits, the deployment of passkeys has also revealed new challenges, particularly around regulatory compliance, fallback mechanisms, and platform lock-in. These are explored in the following section.

2.4.4. Challenges and Issues

While FIDO2 and passkeys present a technically sound and user-friendly alternative to traditional authentication methods, their adoption in real-world environments remains constrained by a combination of technical, organizational, regulatory, and perceptual challenges.

One of the most cited technical limitations is the lack of standardized fallback and recovery mechanisms. Because passkeys (and FIDO credentials in general) are cryptographically bound to user devices, the loss of a device can result in the loss of access to all associated services, unless a backup authenticator has been registered in advance. In Lassak et al.'s (2024) study of enterprise deployments, organizations expressed hesitation about relying solely on passkeys due to concerns over recovery in edge cases, such as lost phones, device resets, or cloud sync failures. Although synchronization across devices mitigates some of this risk, it also introduces a dependency on cloud infrastructure, which may not meet all regulatory requirements, especially in sectors requiring strict local key control.

A related concern is cross-platform and cross-vendor interoperability. While passkeys are supported across Apple, Google, and Microsoft ecosystems, subtle differences in implementation (e.g., credential discoverability, biometric prompts, backup flows) can lead to user confusion. Developers also report difficulties in integrating passkey support into existing applications, particularly when juggling different behaviours across browsers, platforms, and device types [15]. This lack of uniformity is especially problematic for global services that require consistent login behaviour across diverse user bases.

Another core challenge is integration with legacy systems. Many enterprise applications are still built around traditional username/password authentication and session management. Transitioning to a model where the user identity is tied to a cryptographically secured key, rather than a password hash, requires reworking login flows, session token handling, identity federation schemes, and user onboarding processes. Organizations lacking dedicated identity teams or modern authentication platforms often find this migration cost prohibitive. [14]

There are also regulatory and policy-related barriers, particularly in Europe. Regulations such as eIDAS impose specific requirements for strong customer authentication, including hardware-backed key storage, non-exportability, and attestation. While passkeys use hardware isolation and end-to-end encryption, their cloud synchronization model can create compliance tensions. For instance, passkeys stored in iCloud may technically leave the device, even if encrypted, which may not satisfy the requirements for Qualified Electronic Signatures or certain high-assurance identity verification scenarios [15].

Lastly, user perception and trust remain major obstacles. Studies have shown that users often misunderstand the privacy and security guarantees of biometric authentication and passkeys. Common misconceptions include the belief that biometric data is stored in the cloud, that passkeys are irreversible or cannot be revoked, or that switching devices will permanently lock users out. These fears, though unfounded, can erode user confidence and limit adoption unless addressed through clear communication and thoughtful user interface design.

In summary, while FIDO2 and passkeys provide a technically elegant solution to the problems of passwords, their adoption is hindered by challenges that extend beyond cryptographic security. Addressing these challenges requires coordinated efforts across platform vendors, developers, regulators, and UX designers to create an ecosystem that is not only secure, but also resilient, interoperable, and intuitively usable.

2.4.5. Prospects

The emergence of passkeys and the steady adoption of FIDO2 standards signal a decisive shift toward a passwordless future, albeit one that will likely unfold gradually and unevenly across sectors. As vendor ecosystems continue to align, and as new protocols improve usability and portability, passwordless solutions are increasingly positioned as not only a technical innovation, but a necessary response to long-standing user frustrations with online identity systems.

A key motivator for this shift is the widespread issue of password fatigue: the mental burden users experience when managing dozens of unique, complex credentials across services. According to recent research, password fatigue leads users to adopt insecure practices such as password reuse, weak variations, and unsafe storage methods (e.g., plaintext files or emails). This ultimately undermines the security of even well-designed password policies. A study notes that users are “exhausted” by the demands of frequent password changes and complex composition rules and often report frustration with repetitive login flows across services. [16]

The study also found that users value security but quickly abandon MFA setups when the experience is too cumbersome. As one participant put it, MFA is often “more annoying than useful”, especially if used across multiple accounts on a daily basis. Passkeys and FIDO2 offer a promising resolution to both problems. By relying on cryptographic signatures instead of passwords and integrating user presence (e.g., biometrics) as the primary authentication factor, passkeys eliminate the need for MFA in many cases. The device becomes both the first and second factor: possession of the device proves ownership, and biometric/PIN verification confirms intent. This model reduces friction while maintaining or even improving security posture.

Recent innovations point toward a more seamless and user-centric authentication ecosystem. For example, at WWDC25, Apple announced several advancements aimed at simplifying passkey adoption and migration. One such feature is automatic passkey upgrades, where a user logging in with a traditional password is silently issued a FIDO2 credential in the background, accompanied by a prompt to save it. This design lowers the barrier to entry by allowing users to transition from passwords to passkeys without interrupting their workflows or requiring new setup processes [17]

Additionally, Apple introduced the standardized endpoint pattern "/.well-known/passkey-endpoints", which allows service providers to advertise passkey support programmatically. This facilitates credential discoverability and account bootstrapping, streamlining the user experience and enabling third-party applications to integrate passkey support more consistently. Perhaps most notably, the Credential Exchange Protocol (CXP) enables secure passkey import and export, addressing one of the main criticisms of early FIDO2 credentials: that they were bound to a single platform or ecosystem. With CXP, users can securely migrate their credentials between iOS, Android, and Windows environments, opening the door to true cross-platform identity portability [17]

These advances reflect a broader effort by platform vendors to harmonize usability and security. Passkeys retain the anti-phishing, device-isolated, and non-reusable properties of FIDO2, but package them in a format that is accessible to non-technical users. The result is a login experience that is often faster and easier than using passwords, while offering dramatically improved security.

Looking ahead, several promising directions are emerging. One is the integration of passkeys into regulated digital identity frameworks, such as eIDAS 2.0, where they may serve as cryptographic primitives within digital wallets, provided they meet hardware-binding and trust-anchor requirements. Another is their potential role in decentralized identity ecosystems, where passkeys could be used to sign or verify verifiable credentials without relying on central identity providers.

In the enterprise space, as identity orchestration platforms like Azure AD and Okta expand passkey support, we may see a shift toward identity-as-a-platform services that abstract away device management and recovery, while still complying with sector-specific regulations.

Yet challenges remain. True global adoption will require addressing concerns around fallback, accessibility, governance, and vendor lock-in. Efforts like CXP and the WebAuthn Level 3 specification are critical in this regard, but standards alone will not suffice. Success will depend on coordinated investment in education, interoperability, and regulatory harmonization, as well as open participation from both public and private actors.

In conclusion, FIDO2 and passkeys represent the most credible path forward for eliminating passwords at scale. They are not just technical protocols, but part of a broader movement toward user-centric, cryptographically assured identity. While adoption may be incremental, the trajectory is clear: the era of shared secrets is drawing to a close, and with it comes a more secure, intuitive, and resilient foundation for digital authentication.

2.5. Decentralised Identities

Decentralised identities have emerged as a powerful alternative to traditional, centralized identity systems. While centralized models rely on trusted authorities such as LDAP directories, national eID frameworks, or social login providers, decentralised identity frameworks aim to give individuals full control over their digital identity. Rather than placing trust in a single institution, these systems use cryptographic proofs and distributed ledger technologies (DLT) —like blockchains— to secure identity data in a transparent and tamper-evident manner. [18]

At the heart of decentralised identity are Decentralised Identifiers (DIDs) and Verifiable Credentials (VCs). DIDs are globally unique identifiers tied to cryptographic key pairs that the user controls, while VCs allow trusted issuers (such as universities, governments, or employers) to cryptographically sign identity claims about a user. With these tools, individuals can prove specific attributes (like age, education, or professional status) without relying on a central identity provider to mediate every transaction.

A notable real-world implementation of decentralised identity principles is Microsoft's ION network [19], which anchors DIDs to the Bitcoin blockchain. In Europe, projects such as the European Blockchain Services Infrastructure (EBSI) and the European Self-Sovereign Identity Framework (ESSIF) pilot DLT-based identity registries for verifiable academic credentials and cross-border public services.

Ethereum, as one of the most advanced and flexible blockchain platforms, has also become a hub for decentralised identity innovation. Ethereum's smart contracts enable programmable, on-chain identity management that goes beyond simple key ownership. Key Ethereum standards relevant to decentralised identity include:

- ERC-725: A proposed standard for on-chain identity contracts. It allows an Ethereum address (or smart contract) to manage and share identity data, acting as a "container" for identifiers and verifiable claims. [20]
- ERC-735: Works in conjunction with ERC-725 to allow claims (attestations by trusted third parties) to be added to an identity contract.
- EIP-4361 (Sign-In with Ethereum, or SIWE): Defines a standard message format and process for authenticating with web services using an Ethereum wallet, effectively turning a blockchain wallet into a decentralised login identity. [21]

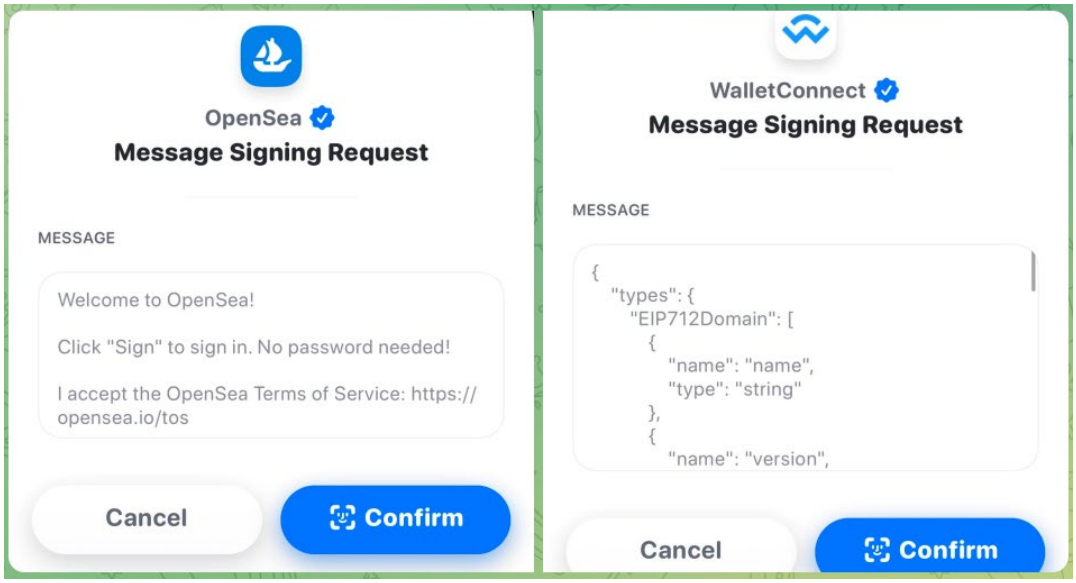


Figure 2.2: Ethereum login message requests [21]

On the left, OpenSea uses a human-readable message (ERC-191 standard [22]), favoured by best practices [21] for user-friendly clarity.
On the right, WalletConnect implements EIP-712 [23], which defines structured, typed data to reduce ambiguity and enhance security.
Both methods leverage cryptographic message signing to authenticate users, reflecting decentralised identity principles.

These standards enable Ethereum wallets, like MetaMask, WalletConnect, or hardware wallets to act as a universal identity layer for web applications (Web3). Instead of usernames and passwords, a user's cryptographic keypair becomes the anchor of their identity. This not only enhances security (no passwords to steal) but also shifts control of digital identity back to users, aligning with the principles of Self-Sovereign Identity (SSI).

However, decentralised identity frameworks also face practical challenges in broader adoption. Key management becomes a significant security concern: loss of a private key can result in loss of the digital identity itself. Interoperability is another major challenge, as different blockchain identity frameworks must integrate with regulated systems like eIDAS to function effectively in real-world scenarios. Finally, while decentralisation maximizes user control and data privacy, many real-world applications, especially those involving e-government or regulated services, require trusted, legally recognized intermediaries to ensure accountability.

The evolution of eIDAS v2 (2024) [7] reflects an attempt to balance these competing priorities. The European Digital Identity Wallet, as defined in eIDAS v2, combines decentralised identity features such as user-controlled credentials and selective disclosure with the requirement for qualified trust service providers to ensure legal validity and compliance. This hybrid approach seeks to incorporate the advantages of decentralised identity while maintaining the robust governance and accountability frameworks needed for secure, cross-border digital interactions.

3. Requirements Analysis

A thorough understanding of system behaviour begins with a clear analysis of what the system must do and how it should operate. This chapter outlines the essential requirements that define the structure and function of the digital identity platform. The analysis begins by identifying key actors (such as users, applications, and administrators) and the processes through which they interact with the system. It then describes the core activities that take place during identity management, registration, authentication, and federation. In addition, a set of operational and behavioural rules are defined to ensure consistent, secure, and compliant system behaviour.

Building upon these elements, the chapter distinguishes between functional requirements, which describe what the system must accomplish, and non-functional requirements, which capture qualities such as security, scalability, and privacy. Together, these components form the foundation for the system's architecture and guide its implementation and evaluation.

3.1. Actors and Business Agents

In the context of a federated digital identity system, business agents are the institutional or technical entities that own, control, or operate key parts of the infrastructure. These include national or regional identity providers, government agencies, trusted service providers, and federation coordinators. Each business agent is responsible for managing a set of users, maintaining identity data, and ensuring compliance with security and privacy regulations. Identity providers (IdPs) act as the gatekeepers of user credentials and are responsible for verifying, authenticating, and issuing identity assertions. A central coordinating agent, such as a gateway operator, ensures interconnectivity between nodes, enabling cross-border or cross-domain interoperability. Blockchain infrastructure may be operated jointly or independently by participating agents, serving as a decentralized layer of trust anchoring identity proofs.

Within the system, various human and system actors interact with components to fulfil operational objectives. The primary human actor is the end user, who registers for a digital identity and uses it to authenticate with applications across different domains. Another key actor is the identity provider administrator, responsible for managing applications, defining access scopes, and configuring identity node behaviour. On the system side, applications (also referred to as relying parties or clients) request authentication and identity data through the identity framework. Identity nodes act as service components that process authentication requests, manage user sessions, and interface with other nodes. Finally, the gateway functions as a federated discovery mechanism, helping resolve the home node for a given user identifier (such as a decentralized identifier or DID).

3.2. Business Rules

To ensure secure, consistent, and privacy-preserving operation of the digital identity system, a set of well-defined business rules governs the behaviour of all components and participants. At the foundation of access control, only applications that are formally registered and approved by an administrator may request identity information. Each of these applications must be assigned one or more authorized scopes, which explicitly define the categories of user data they are permitted to access. The identity information shared during authentication must strictly correspond to these scopes, preventing overexposure of personal data. Furthermore, applications are only allowed to complete authentication flows if their redirect URIs are both valid and transmitted over secure protocols.

Federated authentication processes are also governed by specific rules. When users attempt to log in from a domain different from their home identity provider, the system must redirect them to their authoritative provider based on their decentralized identifier (DID). Identity providers are required to validate the authenticity of any identity tokens received from other nodes before granting access, and they must only respond to authentication requests for which they are authoritative. To maintain strict data boundaries, each identity provider must isolate its user data from other nodes, ensuring that identities are managed independently. Users must explicitly give their consent before any personal identity data is shared with an external application, and all communications involving user identity, whether between identity nodes or to applications, must be conducted over secure, authenticated channels. Additionally, identity assertions must be protected against replay attacks by including both a timestamp and an expiration value. To preserve uniqueness and avoid identifier collisions, each user's DID must be globally unique across all participating providers in the federation.

Together, these business rules create a robust framework for identity management. They establish a clear boundary of responsibilities between applications, identity providers, and infrastructure services, while reinforcing critical principles of security, interoperability, and user autonomy.

Table 3.1: Business Rules Table

Code	Rule
BR1	Only registered applications are allowed to request identity information.
BR2	Each application must be assigned one or more authorized scopes defining accessible data.
BR3	Identity providers must respond only to authentication requests for which they are authoritative.
BR4	Identity information shared with applications must match the scopes granted at registration.

BR5	Federated login must redirect users to their home identity provider based on their DID.
BR6	Only applications with valid redirect URIs may complete authentication flows.
BR7	Identity providers must validate the authenticity of tokens received from other federated nodes.
BR8	Each identity provider must maintain isolation of its user data from other providers.
BR9	Applications must undergo approval by an administrator before being activated in the registry.
BR10	Redirect URIs must use secure protocols (HTTPS) to be considered valid.
BR11	Users must explicitly consent before their identity data is shared with an external application.
BR12	A user's DID must be globally unique across all identity providers in the federation.
BR13	All identity assertions must include a timestamp and expiration to prevent reuse.
BR14	User identity data must only be transmitted between nodes or to applications over secure and authenticated channels.

3.3. Use Cases

The use case tables presented in this chapter provide a structured overview of the key interactions within the digital identity system, detailing how various actors engage with system components to achieve functional goals. Each table follows a standardized format, capturing essential elements such as the use case ID, name, description, involved actors, preconditions, postconditions, main and alternate flows, frequency of occurrence, and priority. This formalization enables clear documentation of both routine and exceptional scenarios, such as user registration, federated login, application scope validation, and node-to-node data exchange. By mapping out these processes in a uniform way, the tables serve as a foundation for system design, testing, and implementation, ensuring that business logic is accurately reflected and consistently applied across the architecture. Furthermore, they provide a useful reference point for identifying security requirements, data flow constraints, and integration dependencies within the federated identity ecosystem.

Table 3.2: Use Case 1 – Register New Application

ID	UC1
Name	Register New Application
Description	Administrator registers a new application for authentication and identity data access.
Actors	Primary: Identity Provider Administrator; Secondary: Application

Preconditions	Admin is authenticated; no duplicate client ID exists.
Postconditions	Application is registered with scopes and redirect URIs.
Main flow	1. Admin accesses registration interface. 2. Enters application details. 3. System validates inputs. 4. Application is stored. 5. Confirmation is shown.
Alternate flows	3a. Duplicate client ID: error. 3b. Invalid input: correction requested.
Frequency	Occasionally
Priority	High

Table 3.3: Use Case 2 - Modify Application Scopes

ID	UC2
Name	Modify Application Scopes
Description	Administrator updates the scopes for an existing application.
Actors	Primary: Identity Provider Administrator; Secondary: Application
Preconditions	Application is registered; admin has access.
Postconditions	Updated scopes are stored and active.
Main flow	1. Admin selects application. 2. Adjusts scopes. 3. System validates and applies changes. 4. Confirmation shown.
Alternate flows	3a. Invalid scopes: error.
Frequency	Occasionally
Priority	Medium

Table 3.4: Use Case 3 - Perform Federated Login

ID	UC3
Name	Perform Federated Login
Description	User logs into an application via their home identity provider using federation.
Actors	Primary: End User; Secondary: Origin IDP, Home IDP, Gateway
Preconditions	User is registered; gateway and nodes are operational.
Postconditions	User is authenticated and receives access token.

Main flow	1. User accesses app. 2. Redirected to foreign IDP. 3. IDP uses gateway to find home. 4. Redirect to home IDP. 5. User authenticates. 6. Token issued and returned. 7. Origin verifies and grants access.
Alternate flows	6a. Auth fails: error. 7a. Token invalid: access denied.
Frequency	Frequently
Priority	High

Table 3.5: Use Case 4 - Register New User

ID	UC4
Name	Register New User
Description	A user creates a new digital identity with their chosen identity provider.
Actors	Primary: End User; Secondary: Identity Provider
Preconditions	User has valid auth method; IDP is available.
Postconditions	New identity is created and optionally anchored on blockchain.
Main flow	1. User opens registration. 2. Submits info and signs proof. 3. System verifies and creates identity. 4. Identity proof optionally stored on blockchain. 5. Confirmation displayed.
Alternate flows	2a. Missing fields: prompt. 3a. Auth fails: abort.
Frequency	Occasionally
Priority	High

Table 3.6: Use Case 5 - Discover User Home Node

ID	UC5
Name	Discover User Home Node
Description	The system determines the user's home identity provider from their DID.
Actors	Primary: Foreign Identity Node; Secondary: Gateway

Preconditions	User presents DID; gateway is operational.
Postconditions	Home node's location is returned.
Main flow	1. Foreign node queries gateway. 2. Gateway forwards to nodes. 3. Positive match returned. 4. Gateway replies with home node info.
Alternate flows	3a. No match: not found response.
Frequency	Frequently
Priority	High

Table 3.7: Use Case 6 – Application Scope Validation

ID	UC6
Name	Application Scope Validation
Description	System validates that applications access only approved user data.
Actors	Primary: Identity Provider; Secondary: Application
Preconditions	App is registered with scopes; user is authenticated.
Postconditions	Token includes only allowed attributes.
Main flow	1. User consents. 2. System checks requested scopes. 3. Validates against allowed scopes. 4. Issues token with permitted data.
Alternate flows	3a. Excessive scopes: denied or trimmed.
Frequency	Always
Priority	High

Table 3.8: Use Case 7 - Node Data Exchange

ID	UC7
Name	Node Data Exchange
Description	Identity nodes exchange identity tokens during federated login.
Actors	Primary: Origin Identity Node; Secondary: Home Identity Node
Preconditions	User is authenticated by home IDP; origin node is awaiting confirmation.

Postconditions	Origin receives token and grants session.
Main flow	1. Home IDP issues token. 2. Sends token to origin. 3. Origin verifies token. 4. Grants access.
Alternate flows	3a. Token invalid: access denied.
Frequency	Frequently
Priority	High

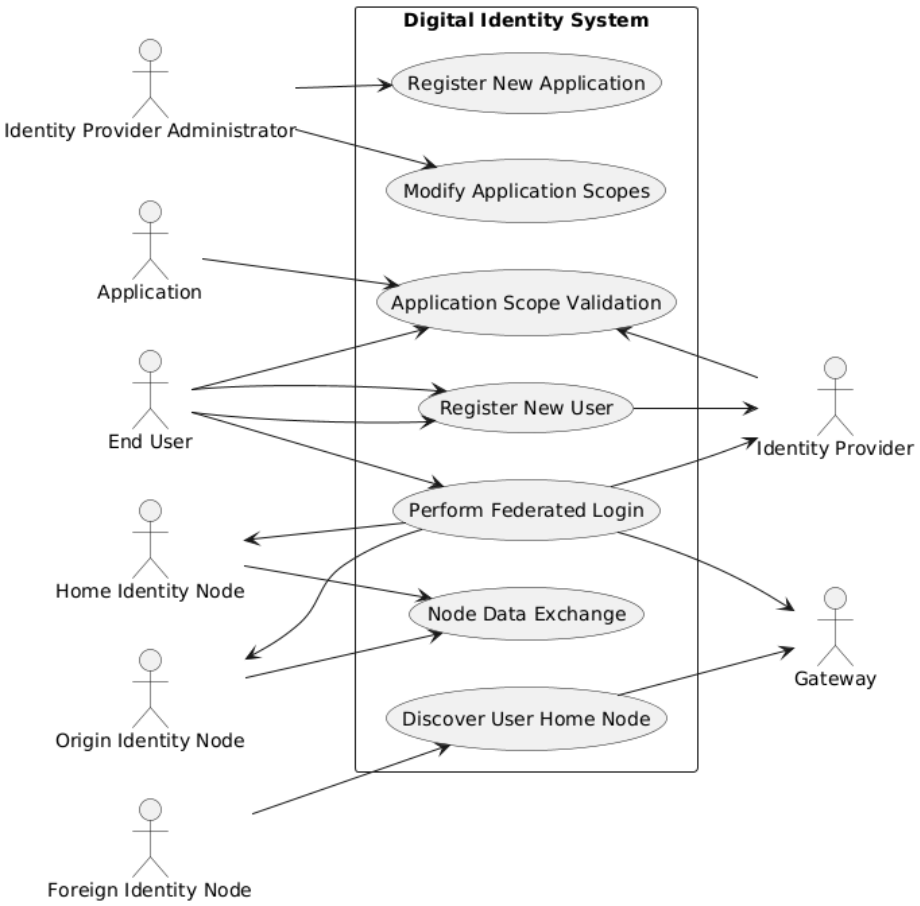


Figure 3.1: Use Case Diagram

3.4. Non-Functional Requirements

Beyond its core operations, the digital identity system must meet several non-functional requirements that govern the quality, security, and reliability of the platform. First and foremost, security is paramount: all communication between components, whether between identity providers, the gateway, or applications, must be encrypted and authenticated to protect against data breaches and impersonation. Identity tokens and user credentials must be handled using industry-standard cryptographic methods, ensuring confidentiality, integrity, and authenticity. Closely related is the requirement for privacy, which mandates strict data

minimization practices, i.e. identity providers must ensure that applications only access attributes explicitly permitted by their registered scopes.

Scalability is another key concern, as the system must be capable of supporting a growing number of users, applications, and participating identity providers without performance degradation. To meet real-world demands, the system should be architected to support at least 100,000 simultaneous users, ensuring responsive performance under peak loads. This requires efficient resource management, non-blocking communication (e.g., for federated discovery), and modular deployment. The system must also ensure availability and reliability, remaining operational even in the event of partial node failure or network disruption.

Deployment complexity is another critical non-functional consideration, particularly given the distributed and federated nature of the system. Coordinating multiple identity providers, gateways, and applications across different environments introduces operational challenges related to configuration, scaling, and dependency management. To mitigate these, the system should leverage containerization technologies such as Docker and orchestration platforms like Kubernetes. Containerization enables consistent environments across development, testing, and production, while orchestration ensures automated scaling, fault tolerance, and service discovery. Moreover, infrastructure-as-code (IaC) approaches and continuous deployment pipelines can further streamline the setup and maintenance of the platform, reducing the risk of human error and enabling rapid, repeatable deployments across diverse infrastructures.

Interoperability is essential in a federated environment, requiring compliance with widely adopted protocols such as OAuth2, OpenID Connect, and DID specifications. The system should facilitate seamless integration across different technologies and jurisdictions. In addition, auditability and maintainability must be supported through comprehensive logging of administrative and authentication events, clear separation of concerns, and consistent design patterns. Finally, the user experience must not be overlooked: authentication and registration flows must be intuitive, responsive, and compatible with modern devices and authentication technologies such as passkeys or crypto wallets. These non-functional requirements ensure the platform operates securely, efficiently, and sustainably in real-world conditions.

4. System Design

THE architecture of the system is designed to model a federated yet extensible identity ecosystem. It consists of multiple autonomous "country systems", each deployed on an isolated virtual machine and equipped with its own “country node”, which hold the application registry, the identity provider, and secure access tunnels. These nodes are connected via a central identity discovery gateway, enabling cross-border authentication and data exchange.

Each country node is implemented as a Spring Boot service and operates independently, reflecting national-level trust boundaries. The nodes are responsible for processing authentication requests, verifying application registration, managing data scopes, and interfacing using a common framework with the other nodes, as well as with their corresponding Identity Providers (IdPs). IdPs differ across countries to demonstrate interoperability between traditional and decentralized identity paradigms: Yellow Country uses a web-based OIDC system based either on a username/password combination or, if enrolled, a passkey, while Orange Country employs an Ethereum-based IdP that verifies wallet signatures and anchors hashed identity claims on the Ethereum blockchain using smart contracts

The discovery gateway plays a pivotal role in coordinating the system. It dynamically determines the user’s home country based on identity hints or user input. This allows the system to overcome one of the core limitations of early federated systems like eIDAS v1, which relied on static mappings of identity providers. The gateway also acts as an intermediary when querying identifiers across nodes, ensuring consistency and interoperability.

Each country node also maintains a local application registry. This registry defines which applications are authorized to request authentication and what specific identity attributes (scopes) they are permitted to access. Applications that wish to support federated login must be pre-registered with the relevant node, along with their allowed scopes and callback URIs. This registry enforces both policy compliance and access control, ensuring that only authorized applications can initiate login flows through that node.

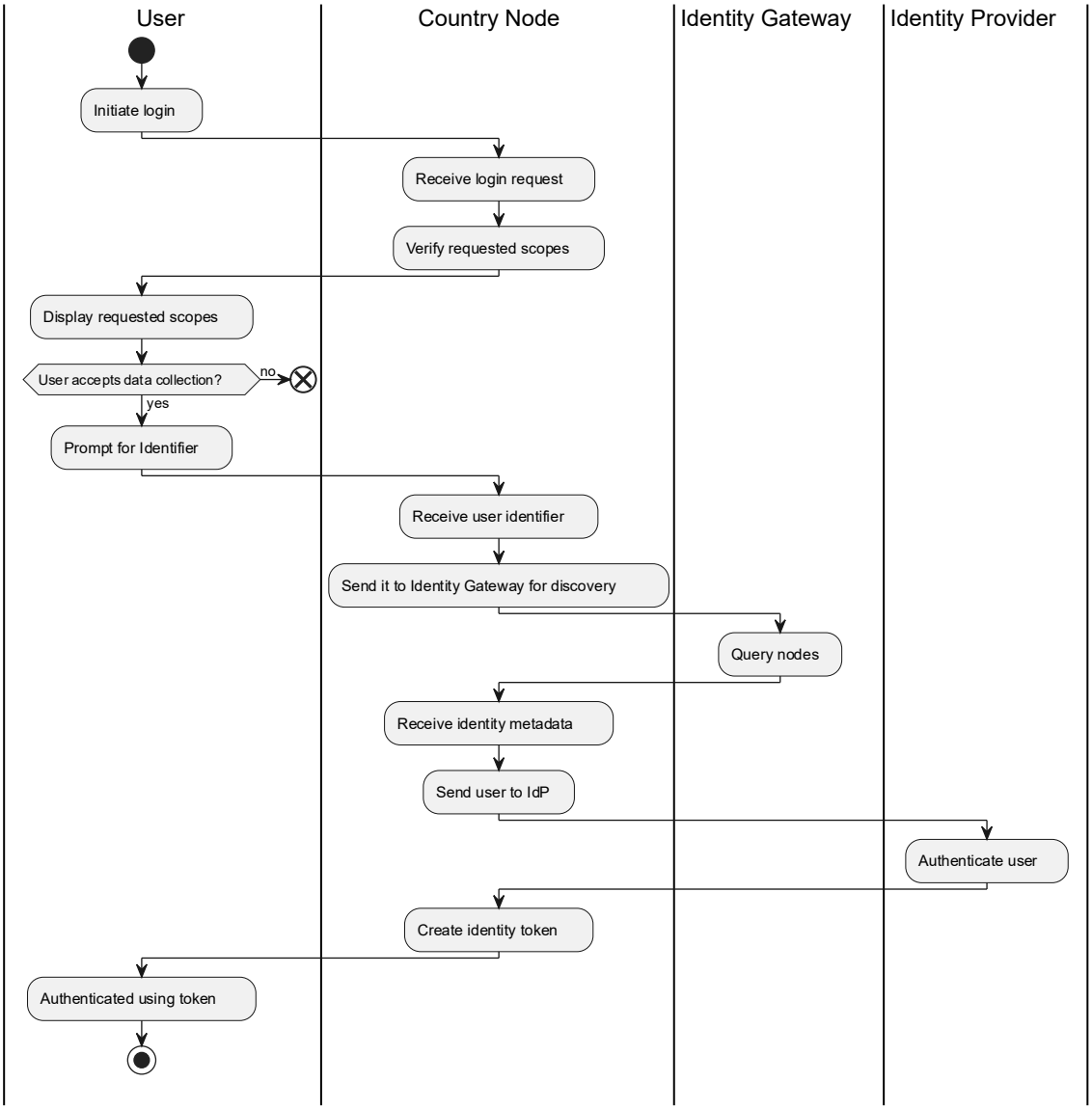


Figure 4.1: Login flow (simplified)

In terms of interaction, the process begins when a user initiates a login on a third-party application. The application redirects the user to the country node it is configured for.

Upon receiving the request, the node first verifies the requesting application's registration and validates the requested data scopes. If approved, the user is presented with a consent interface, where they are shown the scopes being requested and are prompted to enter their identifier.

The identifier can be a DID (Decentralized Identifier) or a traditional username. For blockchain users, this field can be automatically filled using a crypto wallet, which provides an authenticated Ethereum address. Once the user submits their identifier, the country node queries the discovery gateway to resolve the identifier and locate the correct identity provider (which may reside on another node). Once resolved, the node forwards the authentication either to the foreign node, which authenticates the user on its IdP on behalf of the "origin node", or to the local IdP.

The IdP performs the identity verification (either by authenticating via OIDC or by verifying a signed message from a user's blockchain wallet) and issues a signed identity token. This token is returned to the originating application through the country node, completing the login process.

Each virtual machine is configured using a cloud-init script, enabling rapid and reproducible deployment. To securely expose services over the internet, all endpoints are published using Cloudflare Tunnels, which eliminate the need for public IP exposure and reduce the attack surface of each node. This setup ensures that the entire system is cloud-native, secure, and easily scalable, while remaining compliant with privacy regulations like the GDPR by maintaining data locality and auditability at the node level.

The entire system is fully containerized, ensuring consistency across deployments and simplifying both development and maintenance. Each country node, identity provider, and support service runs in isolated Docker containers. These containers are orchestrated at the virtual machine level, enabling modular deployment and strict resource separation between countries. To support maintainability and reproducibility, the system includes a semi-automated CI/CD pipeline: upon new commits or updates to the project's Git repositories, the latest code can be pulled into the virtual machines and automatically rebuild the containers using a multi-stage build process. This approach ensures that all services are deployed with minimal manual intervention while preserving full control over the update process. The combination of containerization and Git-based builds enables a lightweight, repeatable deployment cycle, making the platform suitable for iterative development, testing, and multi-country scaling scenarios.

In sum, this system architecture demonstrates a hybrid identity framework that is capable of bridging centralized trust models and decentralized, blockchain-based identities. It supports both legacy and emerging authentication standards while preserving interoperability, user control, and scalability, characteristics that will be increasingly important in the context of evolving digital identity regulations such as eIDAS v2.

5. Implementation

THIS section discusses the implementation choices made in this project to address the evolving challenges and opportunities in decentralized and federated identity management. The approach draws on lessons from traditional identity frameworks, such as LDAP and SSO, as well as from more modern paradigms like OAuth 2.0, OpenID Connect, and SAML. In parallel, it incorporates insights from decentralized identity standards (like DIDs and Verifiable Credentials) and blockchain ecosystems (notably Ethereum's wallet-based authentication).

A key motivation behind this work was to reconcile the centralized trust models of traditional systems with the user-centric control of decentralized identity. In traditional identity ecosystems, trust is placed in a small number of centralized providers, which can create single points of failure and introduce significant privacy concerns. Decentralized identity approaches, by contrast, rely on cryptographic keys and distributed ledgers to anchor trust, shifting the balance of power to the user.

Yet these paradigms are not mutually exclusive. The hybrid model implemented demonstrates how these approaches can be integrated to create a more robust, interoperable, and user-centric identity ecosystem. In many ways, it reflects the trajectory of the eIDAS framework itself, which has evolved from a purely centralized identity framework (in its first version) to one that recognizes the potential of decentralized and user-controlled identity (in eIDAS v2).

This architecture also underscores the practical challenges in bridging these models: ensuring cross-border interoperability, respecting diverse national regulations, managing technical complexity, and balancing decentralization with legal accountability.

5.1. Identity Gateway

In a decentralized and federated identity system, where users and applications may belong to different administrative domains (or "countries"), the discovery of a user's home identity provider becomes a foundational requirement. The Identity Gateway was developed to address this challenge by enabling dynamic, privacy-preserving routing between country nodes. Its function is intentionally minimal and focused: it does not authenticate users or process any personal data but instead resolves the node responsible for a given identifier, acting as a stateless discovery and delegation service. This design allows trust relationships between countries to be configured once globally, rather than requiring each node to establish and manage bilateral connections with every other participant (unlike traditional frameworks such as eIDAS, which rely on an exhaustive mesh of direct peerings to exchange user information).

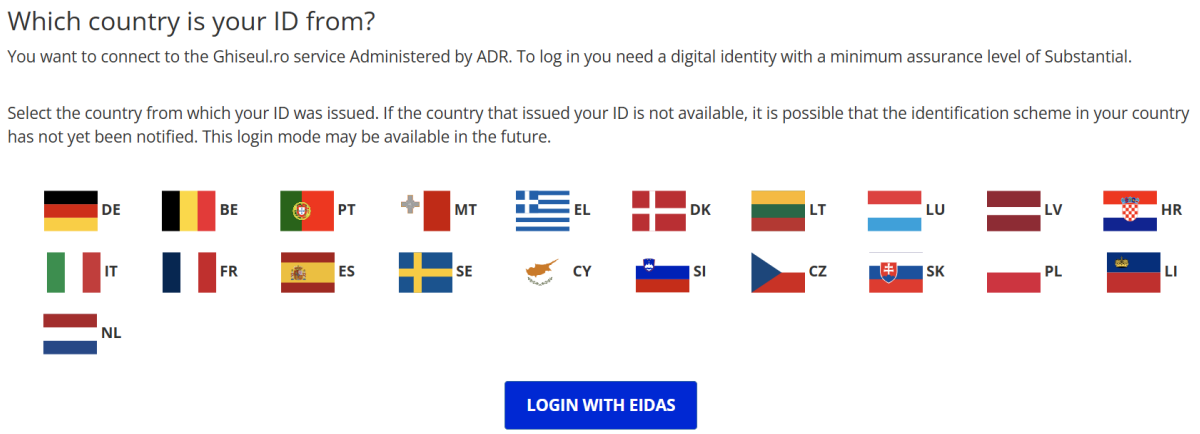


Figure 5.1: Ghiseul.ro eIDAS Login Page [24]

To facilitate stateless, traceable communication between nodes and the gateway, the system defines a set of custom HTTP headers. These headers are used to propagate context, enforce routing logic, and identify the origin and destination of inter-node requests during the authentication discovery process. The following table summarizes the purpose and flow direction of each custom header used throughout the system:

Table 5.1: Custom HTTP Headers Used

Header Name	Purpose
`X-Origin-Node`	Identifies the node that initiated the request (e.g., the application's node). Used for routing, verification, and traceability.
`X-Identity-Node`	Indicates which node owns the user's identifier (e.g., DID or username). Used by the gateway in responses.
`X-Identity-URI`	Provides the full URI of the identity node, allowing the origin node to initiate a proxy login.
`X-Origin-URI`	Echoes back the URI of the requesting (origin) node, useful for mutual redirection and validation.

The gateway is implemented in Spring Boot using Spring WebFlux, allowing reactive, non-blocking parallel HTTP calls to be made to each configured node. The requestor is authenticated using the X-Origin-Node header, which is validated against a preconfigured trust list. The gateway (at this stage) is not publicly accessible; it is only reachable from within the internal virtual machine network, which includes the containerized environments of participating nodes. This minimizes the attack surface and simplifies security for this stage of the project.

When a user initiates a login on an application registered in a local country node (the origin node), their identifier is sent to the gateway. The gateway, in turn, performs a parallel query to all known country nodes, each of which checks if the identifier exists in its local identity provider.

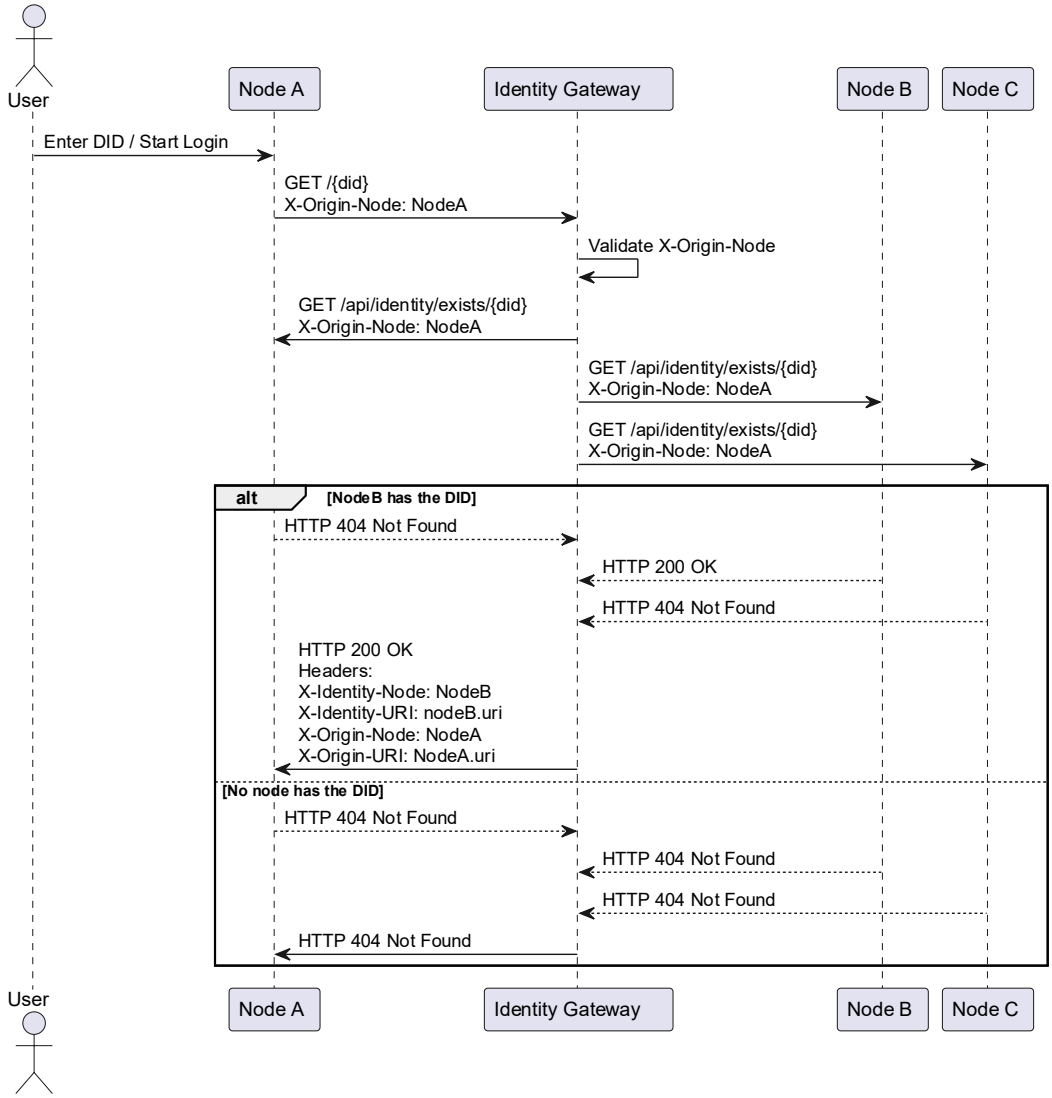


Figure 5.2: Gateway discovery sequence diagram

Note: Although each node can implement filtering logic to block or ignore gateway queries based on the origin node's identity (e.g., for jurisdictional or policy reasons), which is deliberately supported by including the X-Origin-Node in the request to the nodes, the default behaviour is to respond to all valid queries. This approach maximizes interoperability across the federation and aligns with the privacy-preserving design of the system, where no user data is exchanged during discovery.

Each node implements a standard API endpoint (e.g., /api/identity/exists/{did}) and responds with HTTP 200 OK if the identifier is known, or 404 Not Found otherwise, without returning any other information. This ensures that no sensitive data is leaked during discovery and that all responses contain strictly metadata. The gateway waits for the first positive match, then constructs a response containing HTTP headers with routing information that enables the origin node to continue the login process.

While the current design relies on static configuration and header-based trust, it provides a clear path for future decentralization and cryptographic hardening. The architecture is intended to evolve toward more robust, scalable mechanisms for discovery and trust verification, approaches which will be discussed later in this paper.

5.2. Country Nodes – Application Registries

Each "country" in the authentication system is represented by a dedicated country node: a self-contained backend service responsible for managing registered applications, handling authentication flows, and acting as the authoritative resolver for locally owned user identifiers. These nodes are deployed in isolated, containerized virtual machines and interoperate through well-defined interfaces, while remaining administratively and technically autonomous.

One of the key strengths of this architecture is its ability to support heterogeneous identity provider implementations. Each country node may integrate with a different backend identity system, such as an OpenID Connect-compatible username/password database, or a decentralized identity provider based on Ethereum smart contracts and wallet signatures. The federation protocol does not constrain internal identity logic, as long as each node adheres to shared discovery and response standards. This design allows each jurisdiction to adopt custom policies, data models, and trust frameworks, while still participating in a common login ecosystem.

All country nodes are implemented using Spring Boot, chosen for its robust support for RESTful APIs, declarative security, and seamless integration with relational databases such as PostgreSQL. Spring Boot also ensures architectural consistency across components, reducing configuration complexity and enabling the reuse of shared libraries across nodes and the gateway. The framework’s strong modularity and support for containerization (e.g., via Docker) further simplify deployment and scaling within cloud-based environments.

Each node serves two core functions, which will be explained below:

1. Managing an application registry that defines which applications may participate in the login ecosystem and what data they are permitted to request.
2. Acting as an identity node, responsible for resolving and authenticating local user identifiers when queried by foreign nodes.

5.2.1. Application Registry

Each country node includes a local application registry, implemented in PostgreSQL and integrated through Spring Boot's JPA layer. The registry defines which applications are authorized to initiate authentication flows, the user data (scopes) they are allowed to request, and to which redirect URIs they may return after successful login. These rules enforce least-privilege access and enable decentralized policy enforcement at the node level.

The registry consists of two main entity classes:

- ApplicationScope – A named category of user data that may be requested by applications.
- Application – Represents a client app with credentials, redirect URIs, and a list of authorized scopes.

The ApplicationScope class defines individual data attributes or groups of attributes that an application may request access to. Each scope is independently identified and documented, allowing user consent screens and backend logic to operate consistently.

Table 5.2: ApplicationScope Class Table

Field	Type	Description
`name`	String	Primary key and unique identifier for the scope. This is the value used in OIDC scope parameters (e.g., email, profile).
`description`	String	Human-readable explanation of what the scope represents. Displayed in consent dialogs.

The Application class models a client application that wants to authenticate users through the country node. It stores credentials, authorization settings, and associations to permitted scopes:

Table 5.3: Application Class Table

Field	**Type**	**Description**
`appId`	String	Internal unique identifier for the application (immutable primary key).

`clientId`	String	Public identifier used in OAuth/OIDC login and token flows. Must be unique.
`clientSecret`	String	Secret used to authenticate the application during token exchange.
`name`	String	Human-readable name of the application, shown to users during login.
`active`	boolean	Indicates whether the application is currently enabled. Disabled apps are denied access.
`redirectUri`	String	Whitelisted URI where the user is redirected after login. Prevents redirect-based attacks.
`scopes`	Set<ApplicationScope>	Many-to-many relation to scopes. Defines what data the app is allowed to request.

The design of the application scope model draws direct inspiration from the OpenID Connect (OIDC) standard, which defines a set of well-known scopes and corresponding user claims to support structured identity assertions across distributed systems. Core scopes such as openid, profile, and email are included in the registry to maintain compatibility with existing OIDC clients and ensure familiar behaviour during authentication and consent flows. These standard scopes are expanded with additional domain-specific attributes (such as did, pin, and age) to support both decentralized identifiers and privacy-preserving claim alternatives not formally defined in OIDC.

Table 5.4: Scopes List

Scope Name	Description
`openid`	OpenID-compatible identifier (DID or username)
`profile`	Basic profile info: first name, last name, date of birth, gender in OIDC compliant format
`email`	User's email address
`did`	Decentralized Identifier (e.g., did:ethr:...)
`pin`	Personal Identification Number
`name`	Full name (aggregated field)
`given_name`	First name / given name

`family_name`	Last name / family name
`birthdate`	Date of birth (ISO format)
`age`	User's age in years, without revealing birth date
`gender`	User’s gender
`address`	Physical or postal address

To ensure a seamless and consistent user experience across all federated country nodes, the system includes a unified, shared frontend login interface. This frontend is implemented using Next.js and exported as a static site using next export, allowing it to be deployed alongside each Spring Boot backend without introducing runtime dependencies. It serves as the user-facing component of the OAuth 2.0 login flow, responsible for displaying application context, collecting identifiers, handling consent, and bridging the transition from discovery to authentication.

The login interface is typically invoked as part of the standard OAuth 2.0 authorization code flow. When a client application initiates authentication via the /oauth2/authorize endpoint, the backend validates the request and determines whether the user is already authenticated. If not, it redirects to the shared login page, served at /login.html, with relevant query parameters including the client_id and requested scope. This marks the beginning of the interactive phase of the flow, in which the user can review requested data permissions, provide their identifier (e.g., username or DID), and explicitly approve or reject the login attempt.

Table 5.5: Country Node - Application Registry Endpoint List

Endpoint	Purpose
`GET /api/apps/{client_id}`	Returns application metadata for consent screen.
`GET /api/apps/{client_id}/scopes`	Returns scopes only, without other app details.

Once redirected, the frontend uses the query parameters to dynamically fetch application metadata from the backend (via /api/apps/{client_id}), including the name of the requesting application, its registered redirect URI, and the list of requested scopes. The requested scopes are then filtered and displayed to the user in plain language, allowing them to understand exactly what information will be shared if they continue.

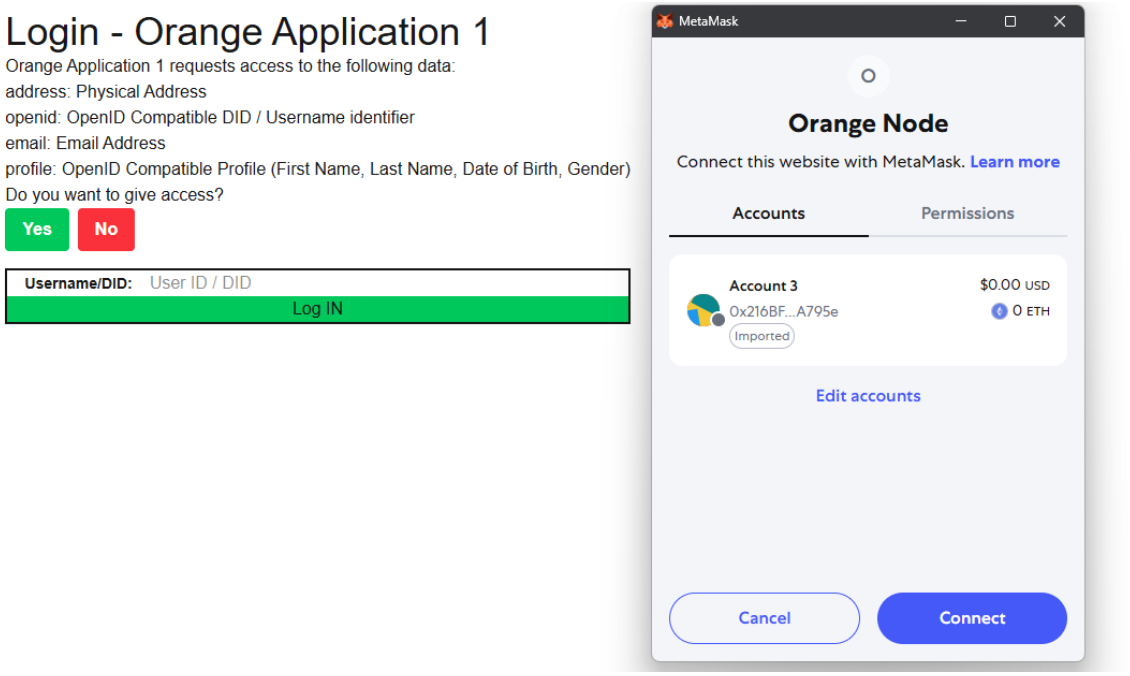


Figure 5.3: Login page MetaMask pre-fill consent form

If the user accepts the request, a form is revealed, prompting them to enter their identifier. If an Ethereum wallet such as MetaMask is detected, the application automatically invokes the eth_requestAccounts method and uses the selected wallet address to prefill a decentralized identifier (DID) using the did:ethr:{address} format. This provides a user-friendly experience for blockchain-based authentication while remaining compatible with traditional user identifiers.

If the user chooses to deny consent for the requested data scopes, the login process is immediately aborted, and no authentication is performed. In this case, the frontend redirects the user back to the client application's redirect_uri with an error query parameter, typically error=access_denied. This behaviour complies with the OAuth 2.0 specification [6] and ensures that users retain full control over their personal information. By explicitly requiring user approval before any identity information is shared, the system enforces a consent-based access model and aligns with principles of data minimization and privacy by design. Applications receiving such an error are responsible for handling it gracefully, for example by informing the user that the login attempt was cancelled or offering the option to retry with reduced permissions.

After the user submits the form, the data is sent to the backend, which continues the authentication process. Upon successful authentication, the node resumes the standard OAuth flow by issuing an authorization code and redirecting the user back to the client application's redirect URI.

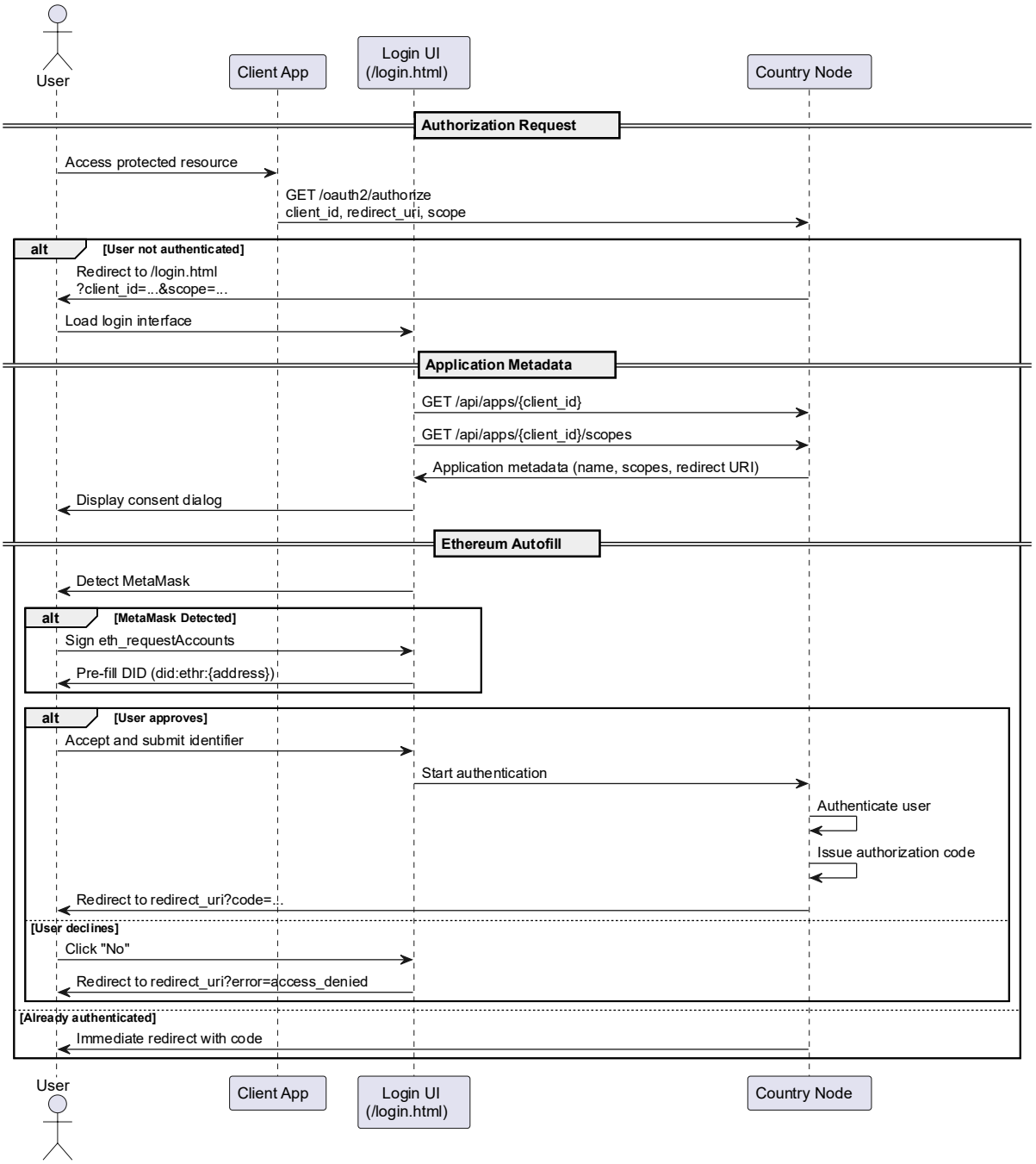


Figure 5.4: User Authorization Flow (Application Registry)

The interface enforces several security measures. It does not expose the client secret or any sensitive internal metadata to the browser. Scope enforcement is mirrored on the server side to prevent unauthorized escalation, and the redirect URI is strictly validated against the application's registry entry to prevent redirection attacks. All communication remains within the same security domain as the backend and is protected by TLS and access control.

This shared login frontend allows for consistent branding, usability, and behaviour across nodes while respecting each node’s autonomy in managing user authentication and identity

logic. Its modularity also allows easy updates; improvements to the interface can be propagated across all nodes by rebuilding their container images with the latest exported static assets.

5.2.2. Identity Node

In addition to managing its local application registry, each country node functions as an identity node: a service that determines whether a user identifier belongs to its jurisdiction, handles local authentication, and coordinates proxy login for users registered in other countries. This role is essential for enabling federated identity without centralization, allowing each node to independently maintain and authenticate its own users while interoperating with the rest of the network.

Table 5.6: Country Node - Identity Node Endpoint List

Endpoint	Purpose
`GET /api/identity/exists/{did}`	Checks whether a user identifier is local to this node.
`POST /api/identity/startLogin`	Initiates login by determining identity ownership and redirecting accordingly.
`GET /api/identity/callback`	Redirect after successful IDP authentication to process identity data.
`GET /api/identity/proxyLogin`	Triggered by origin node to delegate login to a foreign identity node.
`GET /api/identity/proxyCallback`	Receives federated identity assertions from a foreign node after proxy login.

When the login flow begins through the /login.html interface, the node receives a POST request at /api/identity/startLogin with two parameters: the user's identifier and the originating application ID. The node calls the Identity Gateway to determine the owner of the identifier and uses custom HTTP headers (X-Identity-Node, X-Identity-URI, etc.) to extract the appropriate destination node metadata.

Depending on where a user and application are registered, the node may perform local authentication or coordinate a proxy login through another node. These two scenarios are distinct in structure and delegation behaviour, but share common patterns of scope enforcement, trust-based routing, and state security.

Case 1: User and Application Registered in the Same Country/Node

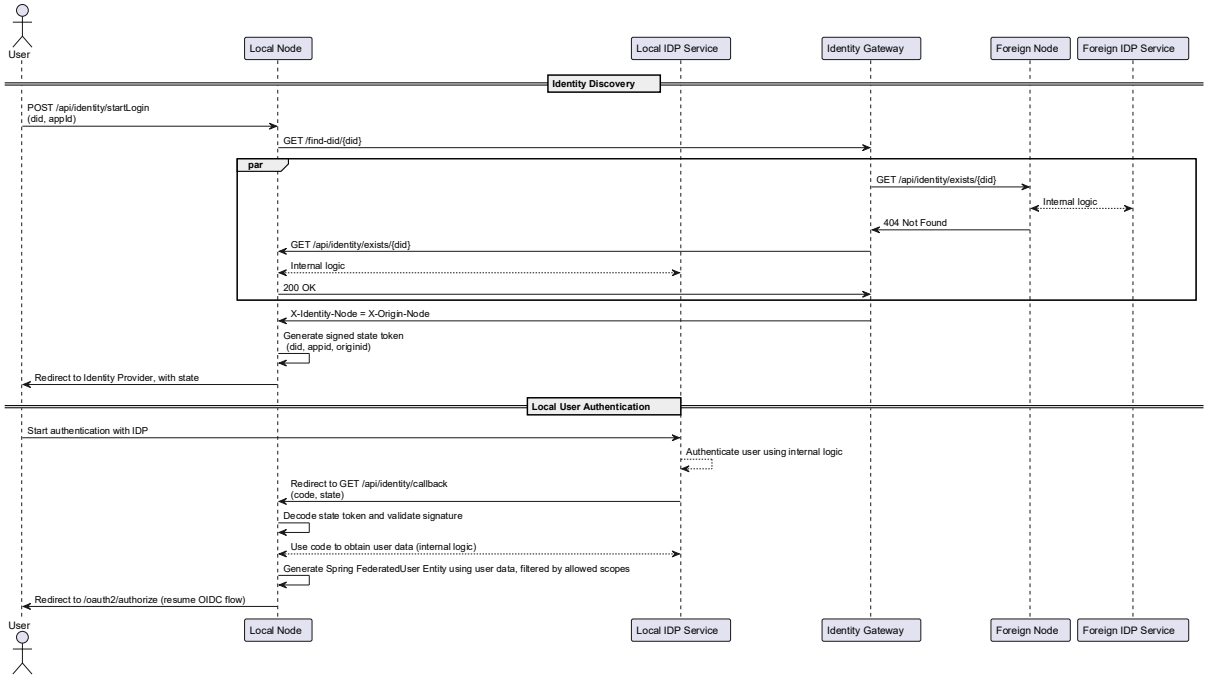


Figure 5.3: Same country authentication sequence diagram.

When both the user and the client application are registered in the same country node, the login process proceeds almost entirely within that country's infrastructure. This flow benefits from minimal latency and full policy control, with no cross-node delegation required. The process can be divided into two main stages: identity discovery and local user authentication.

The interaction begins when the user submits their identifier and grants consent via the shared login frontend. This form submission triggers a POST request to the local node's /api/identity/startLogin endpoint, including the decentralized identifier (DID) and the client application's identifier. To determine where the user is registered, the node queries the Identity Gateway using a GET request to /find-did/{did}.

The Identity Gateway initiates parallel resolution queries by invoking /api/identity/exists/{did} on all known nodes. In the local flow, the foreign node (NodeB) responds with a 404 Not Found after checking internally with its identity provider, indicating that the user is not registered there. The local node (NodeA), in contrast, confirms the user's presence with a 200 OK, after verifying the DID internally via its own identity provider.

Once the gateway determines that the identity is owned by the same node that received the request, it returns confirmation via HTTP headers such as X-Identity-Node. This completes the identity discovery phase. At this point, NodeA proceeds to generate a signed state token. This

state is a blind structure encoded with the application ID, DID, origin node name, and origin URI. It is signed using HMAC-SHA256 to prevent tampering and to authenticate the integrity of the session metadata. Importantly, this state can only be verified by the node that created it.

The user is then redirected to the local node's identity provider, typically implemented as an OpenID Connect (OIDC) authorization endpoint or a blockchain-based wallet authentication UI. This marks the beginning of the authentication phase.

The user is then redirected to the local node's identity provider. Each country node integrates an independent identity provider implementation, designed to support authentication within its jurisdiction. These identity providers are accessed via a common framework, which allows them to interact consistently with the rest of the system despite potentially differing backend technologies.

The authentication phase begins once the user reaches the identity provider interface. After verifying their identity using the mechanisms supported by the local system, the user is redirected back to the node's designated callback endpoint at /api/identity/callback. Crucially, the redirect includes the original state parameter, which carries essential metadata about the login request. Because the callback endpoint itself is stateless, this state token is the only way to securely link the incoming authentication response to the original session context. It ensures that the response cannot be tampered with or misrouted, and that the request is bound to its initiating application and node.

The node begins by validating the received state parameter, verifying its HMAC signature and decoding the embedded metadata to recover the application ID, origin information, and whether the session is local or proxied. Since the /callback endpoint is designed to be stateless, this state token is essential for determining how the login request should be handled and ensuring its integrity. Once validated, the received authorization code is used to query the identity provider for user information. Because each country may implement its identity provider differently, the node is responsible for translating the returned user data into a normalized structure that conforms to the shared system-wide identity format. This ensures that downstream services and authorization logic can operate on a consistent claim set. After translation, the node filters the normalized claims against the application's registered scopes, releasing only those that were explicitly authorized. This enforces strict data minimization, ensuring that users' personal data is never shared beyond what is necessary for the specific service context.

A Spring Security session is established using a FederatedUser principal, which encapsulates the subject (identifier) and the filtered claim set. This authenticated session is stored in the HTTP context for future OAuth processing.

Finally, the original OAuth 2.0 /oauth2/authorize flow is resumed. The node retrieves the saved authorization request from the session, generates an authorization code, and redirects the user back to the application's redirect_uri.

This localized login flow is efficient, secure, and privacy-respecting. It ensures full enforcement of scope policies, internalizes session creation, and allows the country node to maintain total control over authentication, claim handling, and user consent.

Case 2: User and Application Registered in the Different Jurisdictions

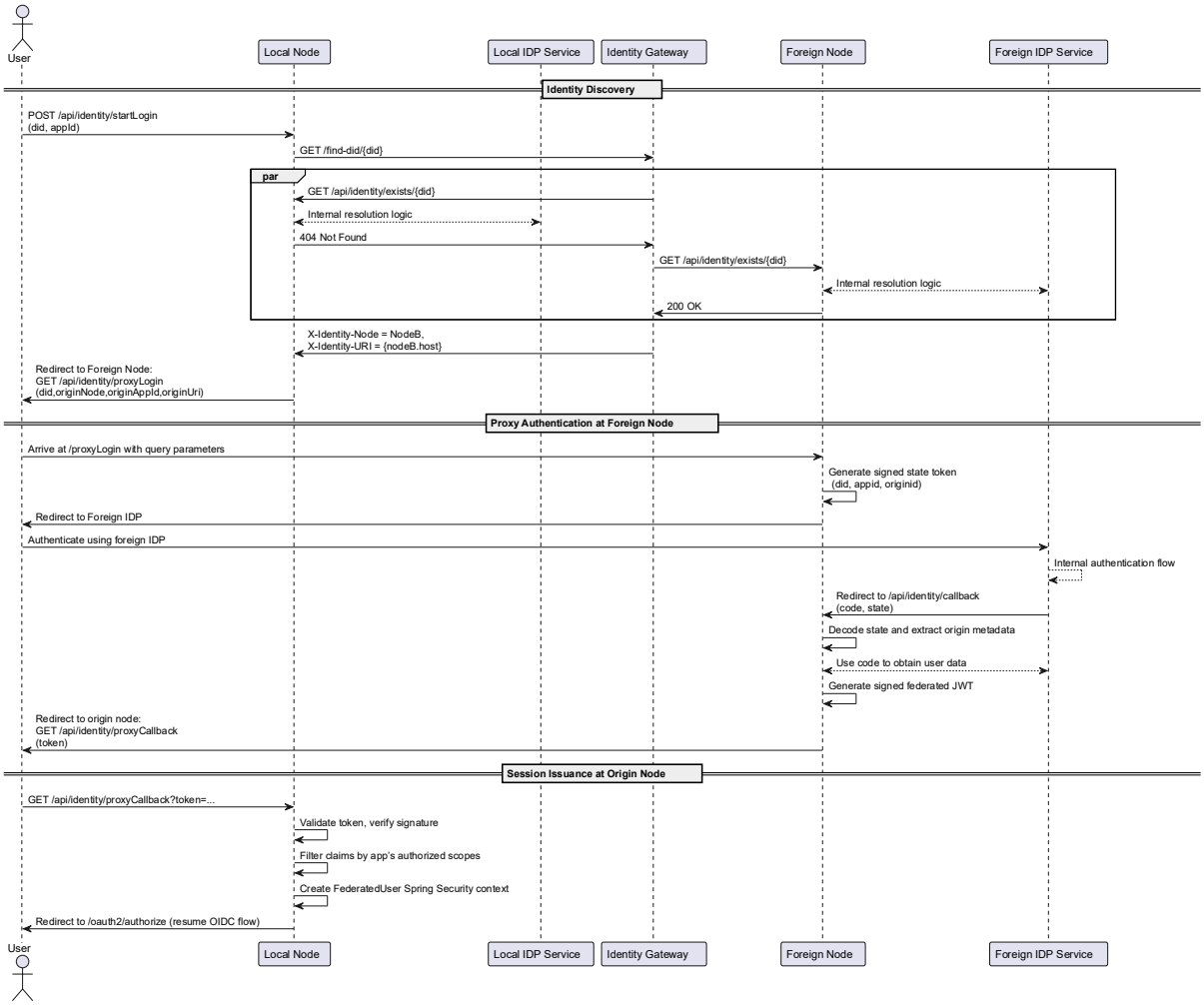


Figure 5.4: Different countries authentication sequence diagram

In cases where a user and the application they are attempting to access are registered in different country nodes, the system performs a proxy login flow. This federated approach ensures that each node retains full control over its own users and authentication logic, while still allowing users to access applications hosted outside their country of origin. This process is divided into three phases: identity discovery, proxy authentication, and session issuance.

The identity discovery phase follows the same mechanism described in the local login flow. The origin node sends the submitted identifier to the Identity Gateway, which queries all participating nodes via their /api/identity/exists/{did} endpoint. The first node to respond with 200 OK is identified as the authoritative source for that identity. In this case, the gateway determines that the user is registered in a different jurisdiction, prompting the origin node to initiate the proxy login sequence.

Upon determining that the user is registered in a foreign node (NodeB), the origin node (NodeA) initiates a redirect to NodeB's /api/identity/proxyLogin endpoint. Rather than passing routing information through URL parameters, the system relies on custom HTTP headers to carry critical metadata, as defined in Table 3.1. These headers include X-Origin-Node (identifying the requesting node), X-Origin-URI (specifying the origin node's host address), and X-Origin-AppId (indicating the client application involved). This design allows nodes to validate the structure and authenticity of incoming requests before continuing with authentication.

Requests with missing or malformed headers or originating from unrecognized/disallowed nodes (according to the foreign node's own policies) can be rejected early, before sensitive authentication logic is engaged. This helps prevent unauthorized nodes from spoofing login requests and enables fine-grained control over federation membership and trust boundaries. While not a cryptographic guarantee on its own, this mechanism acts as a lightweight trust layer, supporting federation integrity and enforcing protocol conformity between participants.

Once the headers are successfully validated, NodeB proceeds to generate a signed state token that anchors the login session. This state is a blind, tamper-resistant artifact that encodes essential metadata such as the original node name, the application ID, and other session context in a base64-encoded string. It is then signed using HMAC with a secret key known only to the foreign node, ensuring that it cannot be forged or modified by external parties. Because the foreign node retains sole authority over the state format and signing key, it can later verify the authenticity of the session when the user returns after authentication.

After generating the state, NodeB redirects the user to its internal identity provider, initiating the authentication phase. Upon successful login, the identity provider returns control to NodeB's /api/identity/callback endpoint, along with the original state and an authorization code. NodeB verifies the state signature to ensure it matches an outstanding session request, then uses the code to retrieve identity data from its local identity provider.

Given that each node may use a distinct format for storing and representing identity data, NodeB is responsible for mapping the identity information into a shared, normalized claim structure compatible with the federated system. It then filters these claims based on the application context and constructs a signed federated JWT, which securely encapsulates the validated identity information. Finally, NodeB redirects the user back to the origin node's /api/identity/proxyCallback endpoint, passing the JWT in a query parameter to complete the proxy login flow.

After the user is redirected back to the origin node (NodeA), the request is received at the /api/identity/proxyCallback endpoint, along with the signed federated JWT generated by the foreign node. This token contains the verified user identity attributes, normalized into the system's shared schema, and serves as the authoritative authentication artifact for the session.

The origin node begins by validating the JWT's integrity and authenticity, using a locally configured shared secret or public key associated with the issuing node. This verification step ensures that the token has not been tampered with and that it was indeed issued by a trusted federation member. If the signature is invalid or the token is expired, the request is rejected.

Once validated, NodeA extracts the identity claims from the token and evaluates them against the set of scopes that were originally requested by the client application. With the authorized identity data in hand, NodeA creates the FederatedUser instance, which is inserted into the current security context and saved using the session repository. Finally, the origin node resumes the previously paused OAuth 2.0 authorization flow. Leveraging Spring Security's request cache, it retrieves the saved request that initiated the login and completes the flow by issuing an authorization code. In both the local and proxy login flows, the final stage of the authentication process is completed by the client application through a call to the /oauth2/token endpoint. This standard OAuth 2.0 endpoint is responsible for exchanging the previously issued authorization code for a set of tokens that establish the user's authenticated session.

Table 5.7: Tokens Issued During OAuth 2.0 Authorization Code Exchange

Token Type	Purpose	Usage
Access Token	Grants permission to access protected resources (e.g., APIs) on the user's behalf.	Sent with API requests as a bearer token in the `Authorization` header. Typically short-lived.
ID Token	Provides verified identity information about the authenticated user.	Used by the client to retrieve user claims (e.g., subject ID, name, email). Follows OIDC format.
Refresh Token	Allows clients to request a new access token without re-authenticating the user.	Sent to the `/oauth2/token` endpoint with grant type `refresh_token` to renew sessions.

The entire exchange is implemented using Spring Authorization Server, which natively supports the OAuth 2.0 and OpenID Connect (OIDC) protocols. To ensure interoperability across the system, the ID token follows a custom OIDC-compatible format, harmonized with the federated claim structure established during the authentication phase. This architecture guarantees that, regardless of whether the login is local or delegated, the session tokens issued to the application are consistent, secure, and standards compliant.

5.3. Identity Providers

Identity Providers (IDPs) are the authentication components embedded within each country node. They are responsible for verifying the identity of users who are registered in that specific jurisdiction. Each node integrates a local IDP implementation that reflects its internal identity policies and technical preferences, ranging from traditional credential-based systems to decentralized identity solutions. Crucially, in this architecture, an IDP is only ever called by its own node, and never directly exposed to or accessed by foreign nodes.

During the login process, the node handles all external coordination. For local users, it delegates authentication to its internal IDP and uses the results to build a secure session. For foreign users, it never interacts with external IDPs directly; instead, the foreign node is contacted and takes full responsibility for invoking its own IDP, authenticating the user, collecting and formatting the identity claims, and returning a signed result to the origin node.

This separation ensures that identity providers remain jurisdictionally isolated, reducing attack surface and enforcing data locality. The interface between a node and its IDP is internally standardized, with endpoints such as /callback, but the implementation behind these endpoints can vary freely. This allows each node to adapt to national standards, preferred authentication factors, or emerging identity technologies, such as blockchain-based wallets or biometric ID verification without impacting interoperability.

To associate the authentication session with the correct login context, the node embeds a signed and opaque state token into the redirect to its IDP. This token encodes metadata such as the origin node, application ID, and user identifier in a format that is cryptographically signed by the node. Importantly, the identity provider itself does not interpret or validate this token; it simply returns it unmodified upon successful authentication.

This system includes two distinct identity provider implementations, each illustrating the flexibility and jurisdictional independence of the architecture. The "Yellow" Country node integrates a traditional OAuth 2.0 login system supporting both username/password authentication and passkey-based flows, reflecting a standards-compliant but centralized model. In contrast, the "Orange" Country node uses a blockchain-based IDP built on the Sepolia Ethereum test network, where user identities are validated through on-chain smart contracts. This implementation relies on the user’s Ethereum wallet and cryptographic signatures to authenticate without revealing personal credentials. Despite these differing backends, both IDPs are accessed through the same shared framework and interface structure, demonstrating how decentralized and conventional identity systems can coexist within a unified federation.

5.3.1.Banana Country

The Banana Country's identity infrastructure is centred around a traditional OAuth2 and OpenID Connect (OIDC) provider implementation known as the Banana IDP. This identity provider is a Node.js-based web service that supports both legacy username/password authentication and modern passkey-based (WebAuthn) login mechanisms. The Banana IDP acts as the internal identity authority for the Banana country node, which handles all proxying and federation responsibilities, while also being a full OAuth2 implementation that can serve identities on its own.

The Banana IDP is implemented using the Express.js web framework, with EJS templates providing the frontend rendering layer. The service utilizes HTTPS and environment-driven configuration and relies on a MySQL relational database to persist identity data. Session management is handled through express-session, and logging is performed using Morgan. It supports two authentication paradigms:

- Username and Password: A traditional login method backed by bcrypt-hashed credentials in the users table. Login UIs are presented through the /sso/login page and supported by CAPTCHA-based abuse protection.
- Passkeys (WebAuthn): Passwordless authentication is provided via integration with the Passwordless.dev API. Passkeys can be created and managed through the /passkeys endpoint and used during login with biometric or PIN-based identity verification. These passkeys are stored remotely but are indexed locally via the user_passkeys table.

Table 5.8: Banana IDP Database Structure

Table	Purpose
`users`	Stores user identity data including username, email, name, DOB, and PIN
`clients`	Registered OAuth2 clients allowed to initiate login flows
`audit_log`	Records login, registration, and account change events
`user_passkeys`	Indexes registered WebAuthn passkeys
`audit_log_actions`	Maps event types (e.g., sign-in, password change)

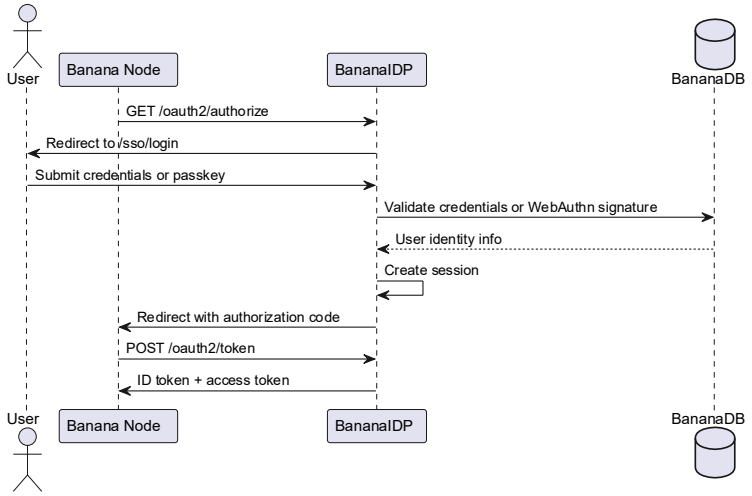


Figure 5.5: Banana IDP Sequence Diagram

Authentication flows begin when a relying party (application) initiates a login by redirecting the user to the /oauth2/authorize endpoint. This endpoint verifies whether a session already exists; if not, the user is redirected to the login page at /sso/login, optionally prefilled with a login_hint. Upon providing credentials or authenticating via a passkey, the IDP verifies the login against the user database, creates a session, and then issues an authorization code to continue the OAuth2 process.

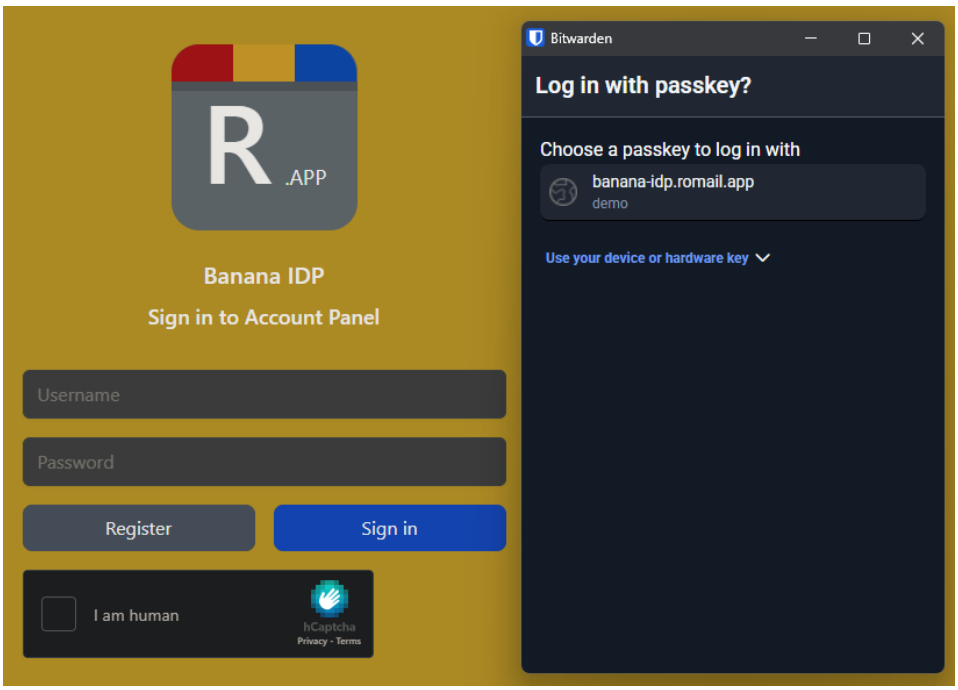


Figure 5.6: Banana Login Interface with Passkey Prompt

This figure illustrates the login interface of the Banana IDP during a passkey-based authentication attempt. On the left, the Banana IDP's customized login page is rendered, offering fields for traditional username and password authentication, a registration button, and

a human verification step via hCaptcha. This fallback ensures compatibility for users not enrolled in passwordless login, while also keeping request secured from automated attacks. Meanwhile, on the right, a native operating system dialog (in this case, managed by Bitwarden) appears as a WebAuthn-compatible prompt asking the user to "Log in with passkey." The dialog shows the available passkey associated with the Banana IDP's domain and the registered username. This prompt is automatically triggered by the browser upon visiting the login page, provided the user has a registered passkey and the browser detects FIDO2 capabilities.

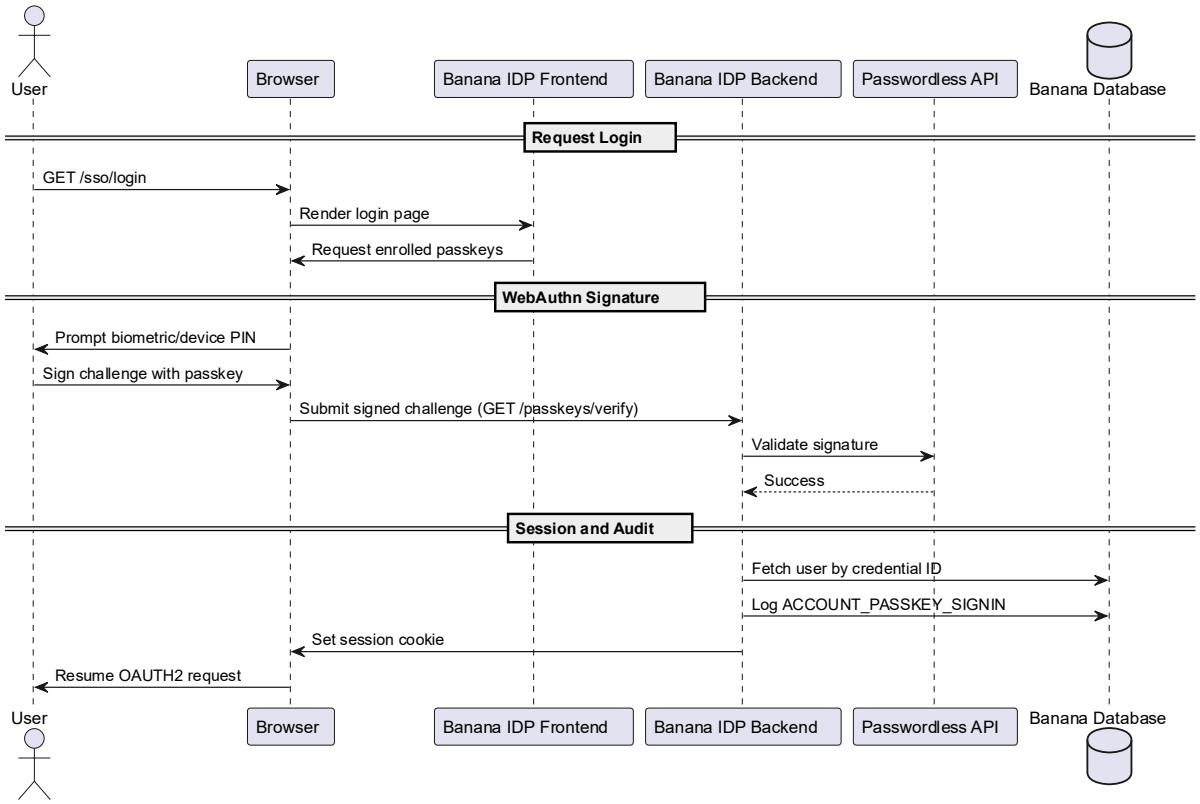


Figure 5.7: Banana IDP Passkey Authentication Flow

This sequence diagram illustrates the end-to-end flow of a passkey (WebAuthn) login process in the Banana IDP, highlighting the interaction between the browser, frontend and backend components of the IDP, the external Passwordless API, and the internal database.

The process begins when the frontend detects passkey support, initiating a challenge by contacting the backend, which in turn requests a cryptographic challenge token from the Passwordless API. This token is returned and passed to the client, preparing the browser to prompt the user for biometric or device-based authentication. Then, the user is issued a challenge, which is signed using their passkey. The signed assertion is sent back to the backend, which forwards it to the Passwordless API for verification. If valid, this confirms that the user controls the device and the registered passkey. Finally, the backend retrieves the user's identity from the database based on the credential ID associated with the passkey. An authentication session is created, a log entry is stored in the audit log, and the user is redirected to the

authorized application or dashboard. This flow demonstrates a secure and seamless login experience that eliminates password-based vulnerabilities and supports strong authentication aligned with modern identity standards.

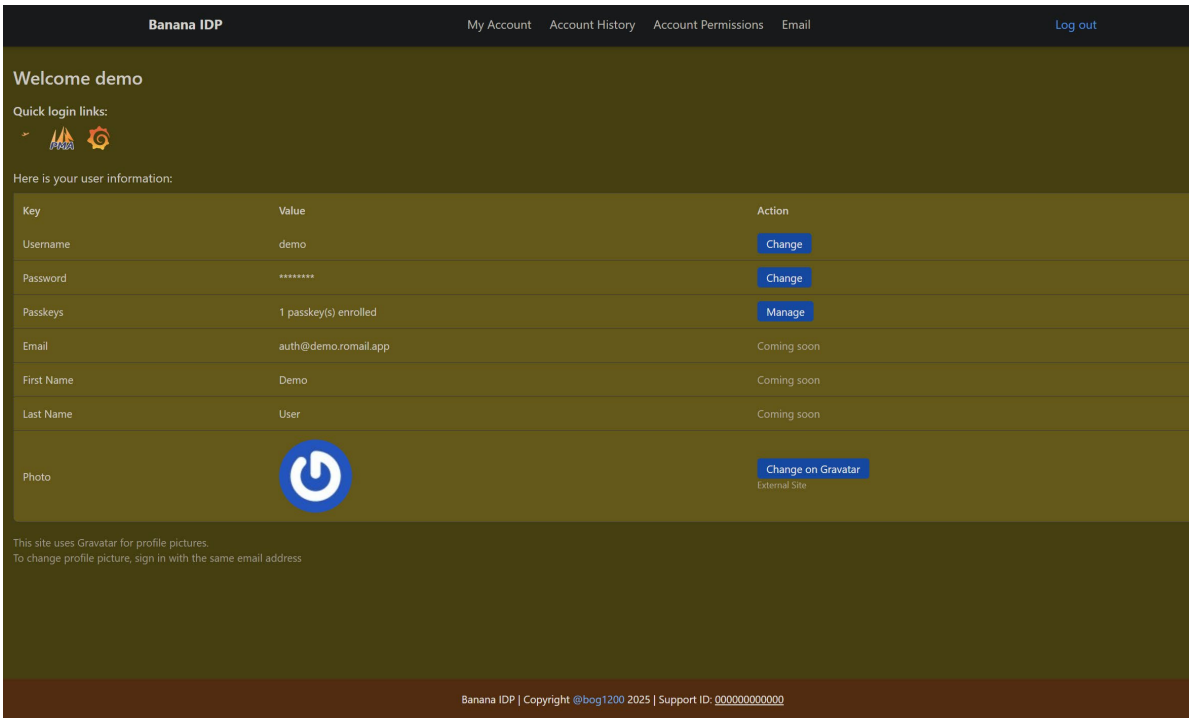


Figure 5.8: Banana IDP Account Page

This figure displays the authenticated user dashboard within the Banana IDP system. After a successful login, users are redirected to this account page, which provides an overview of their profile data, credential settings, and linked authentication methods. At the top of the interface, the system greets the user by their display name and provides "Quick login links" to affiliated services, allowing seamless navigation between connected applications. Meanwhile, the central table lists key identity attributes, including the username, email, first and last names, and profile photo. The user's avatar is displayed via Gravatar, with the current image derived from the email address hash.

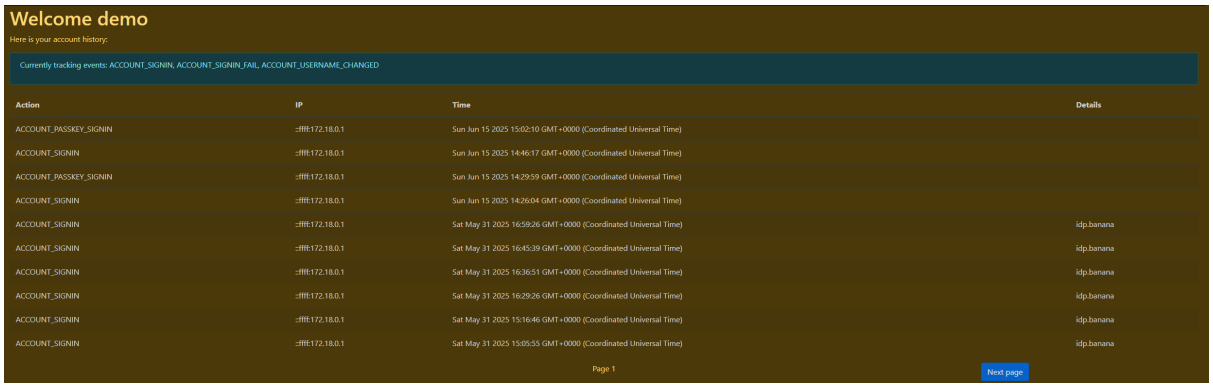


Figure 5.9: Banana IDP Audit Log Page

Here the audit log interface is shown within the Banana IDP user account portal. It provides a chronological record of tracked identity-related events, enabling users to monitor their login history and detect any unauthorized access attempts. The header notes which event types are currently logged, offering transparency about what activity is visible to users.

Each log entry is displayed in a table format, with columns for the event type (Action), source IP address (IP), timestamp (Time), and associated metadata (Details). The interface includes pagination controls (e.g., "Next page"), enabling navigation through historical data. This aligns with the underlying database schema, where logs are indexed and queried in batches to optimize performance.

By surfacing this data directly to users, the Banana IDP fosters trust, improves security hygiene, and empowers individuals to notice and report irregular account activity. This interface demonstrates the project's alignment with the principles of auditability and accountability in digital identity systems.

Table 5.9: Banana IDP Audit Log Events

Event	Description	Monitored
ACCOUNT_CREATED	User successfully created a new account.	NO
ACCOUNT_EMAIL_CHANGED	User changed their registered email address.	NO
ACCOUNT_NAME_CHANGED	User updated their first or last name.	NO
ACCOUNT_PASSKEY_CREATED	A new WebAuthn passkey was registered for the user.	NO
ACCOUNT_PASSKEY_SIGNIN	User authenticated using a WebAuthn passkey.	YES
ACCOUNT_PASS_CHANGED	User changed their account password.	NO
ACCOUNT_SIGNIN	Successful login with username and password.	YES
ACCOUNT_SIGNIN_FAIL	Failed login attempt with incorrect credentials.	YES
ACCOUNT_USERNAME_CHANGED	User updated their account username.	YES

Table 5.10: Banana IDP Endpoint List

Endpoint	Purpose
`GET /sso/login`	Renders the login page with optional `login_hint` and client metadata.
`POST /sso/login`	Handles user login with username/password.
`GET /sso/register`	Displays the registration form.
`POST /sso/register`	Processes user registration.
`GET /sso/logout`	Logs out the current session.
`POST /sso/verifytoken`	Redirects to `/oauth2/token` for token exchange.
`GET /sso/verifyDID`	Checks if a username exists in the database.
`GET /oauth2/authorize`	Entry point for OAuth2 login flow; handles login and redirect with code.
`POST /oauth2/token`	Exchanges authorization code for access token
`GET /oauth2/userinfo`	Returns user claims based on access token.
`GET /account`	Main user dashboard.
`GET /account/history`	Displays the user's recent audit log entries.
`GET /account/passkeys`	Page to view and manage WebAuthn passkeys.
`POST /account/change_username`	Updates the user's username.
`POST /account/change_password`	Updates the user's password.
`GET /passkeys/create`	Generates a registration token for creating a passkey.
`GET /passkeys/list`	Retrieves a list of the user's registered passkeys.
`GET /passkeys/count`	Returns the number of passkeys the user has.
`GET /passkeys/verify`	Validates a passkey login token with the Passwordless API.
`GET /.well-known/openid-configuration`	Provides OIDC discovery document.
`GET /.well-known/jwks.json`	Provides the IDP’s public keys in JWKS format.
`GET /static/public.key`	The RSA Public Key used to sign the JWT Tokens

5.3.2.Orange Country

The Orange Country identity provider represents a novel approach to user authentication and digital identity management by leveraging blockchain technologies, specifically Ethereum smart contracts, and a fully modernized web application architecture. This implementation was designed to contrast with traditional federated login systems, instead offering a decentralized login experience where identity proofs are cryptographically anchored on the public Sepolia Ethereum test network.

To support this architecture, the Orange Country IDP was implemented using Next.js as the application framework and MongoDB as the underlying database. These technologies were selected not only for their technical capabilities but also for their alignment with the system's goals of decentralization, scalability, and developer efficiency.

Next.js, a React-based full-stack framework, provides a seamless development model for building interactive user interfaces, server-side APIs, and static assets under a unified structure. Its integration with the React ecosystem allows for a fluid and dynamic user experience while maintaining server-side security through API route endpoints. In the Orange Country IDP, Next.js powers the login, registration, and verification interfaces. These pages facilitate interaction with Ethereum wallets like MetaMask, guide users through signing challenges, and handle redirects as part of the login and federated authentication flow. The use of TypeScript, built-in routing, and component-based design contributes to better maintainability, development speed, and type safety in sensitive cryptographic operations.

MongoDB, on the other hand, supports flexible and scalable storage of identity profiles. As a document-oriented NoSQL database, MongoDB accommodates dynamic user attributes and avoids the rigidity of a fixed relational schema, an advantage when storing decentralized identifiers (DIDs), associated attributes (e.g., name, gender, address), and temporary cryptographic challenges (nonces). Its JSON-like structure maps naturally to the identity objects processed by the application. MongoDB’s performance characteristics and support for asynchronous data access fit well with the modern, non-blocking architecture of Next.js, allowing the system to serve both user data and ephemeral authentication information efficiently and reliably.

Together, this architecture enables the Orange Country IDP to act as a decentralized identity issuer that is both self-contained and interoperable. The system stores the hash of identity claims (not the claims themselves) on the Ethereum blockchain via a smart contract, offering transparency, immutability, and a path to external verifiability. Meanwhile, the full identity profile remains stored off-chain in MongoDB and is only revealed to authorized nodes after proper cryptographic proof of ownership.

This approach exemplifies a hybrid architecture, where blockchain is used for integrity and proof anchoring, while conventional databases are used for secure and performant off-chain data storage. This division of responsibility allows Orange Country’s IDP to remain

lightweight, secure, and interoperable with the federated network of nodes described in the broader system.

In the following sections, the login and registration flows will be explained in detail, explore the smart contract design, and outline the web-based user experience that powers the Orange IDP.

Table 5.11: Orange IDP Endpoint List

Endpoint	Description
`GET /login`	Login page for walled-based authentication
`GET /register`	Registration form for identity enrolment
`GET /verify`	Displays identity verification based on blockchain and database comparison
`POST /api/auth/nonce`	Generates and stores a nonce for DID authentication
`POST /api/auth/login`	Verifies a signed nonce and issues a JWT token
`POST /api/auth/register`	Registers a new user and returns identity hash
`GET /api/users/{did}`	Checks if a user exists for a given DID
`GET /api/users/{did}/userinfo`	Returns user profile after signature verification

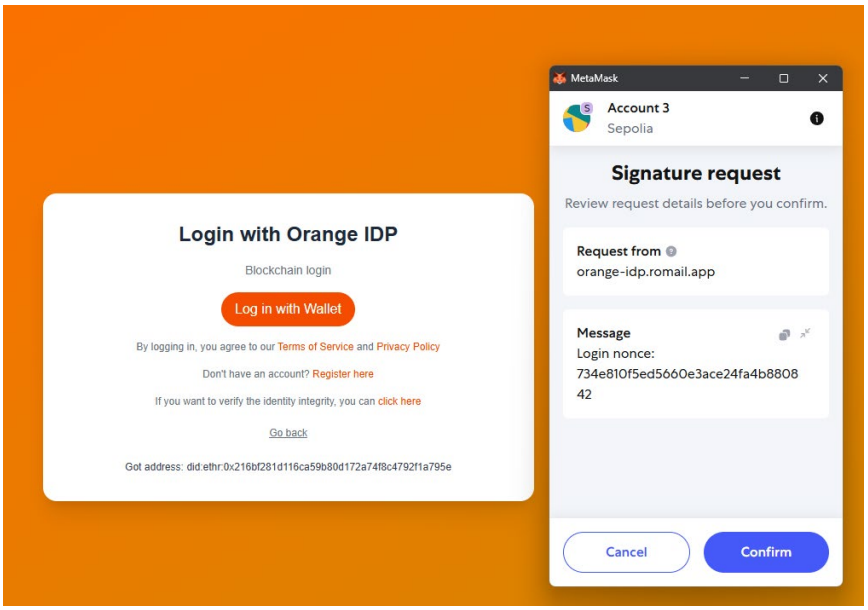


Figure 5.10: Orange IDP Login Page with Signature Request

At the heart of the system is a cryptographic challenge–response mechanism, implemented through a nonce-based Ethereum wallet login flow. When a user attempts to authenticate or register, they first provide a Decentralized Identifier (DID), formatted as did:ethr:{address}. The system then generates a random nonce, stores it temporarily in the MongoDB-backed

Nonce collection, and returns it to the frontend. The user is prompted to digitally sign this nonce using their private key via MetaMask or another wallet interface.

This signature serves as proof of control over the associated Ethereum address. On the backend, the application uses ethers.js to verify the authenticity of the signature. If successful, the DID is considered valid and tied to the login or registration session. This stateless verification mechanism eliminates the need for passwords entirely, offering a user experience comparable to modern passkey systems, anchored in a cryptographically secure, decentralized identity scheme. Once the user's identity has been validated, the IDP issues a JSON Web Token (JWT) that encodes all relevant user claims, such as name, email, date of birth, gender, and more. This token serves as the standardized identity container used in federated authentication flows with the country node and other systems. JWT signing is performed with a shared secret, and expiration control ensures session safety.

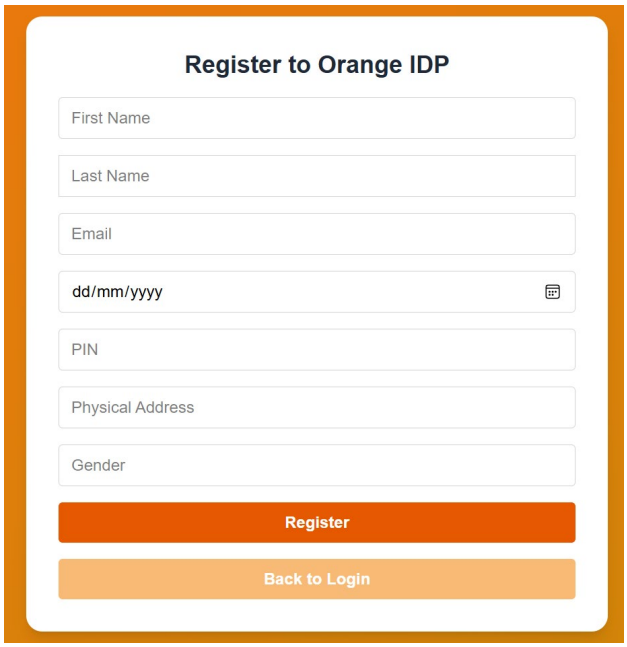


Figure 5.11: Orange IDP Registration Form

The registration flow further includes a blockchain anchoring step. After successful identity verification, the user's identity attributes are encoded and hashed using keccak256 and Ethereum's AbiCoder, ensuring a consistent and deterministic fingerprint of the identity data. This hash, along with the DID, is then submitted to a smart contract deployed on the Sepolia testnet, a public Ethereum-compatible network. The OrangeIDP contract stores mappings from user addresses to hashed identities and emits on-chain events to signal new registrations or updates. Because the on-chain data contains only a cryptographic digest (not the identity itself), privacy is preserved, while still enabling public verifiability. Any relying party can independently verify the integrity of an identity profile by hashing the user's current data and comparing it to the value stored on-chain. This design balances data minimization, user control, and verifiability, all while enabling secure interoperability across jurisdictions.

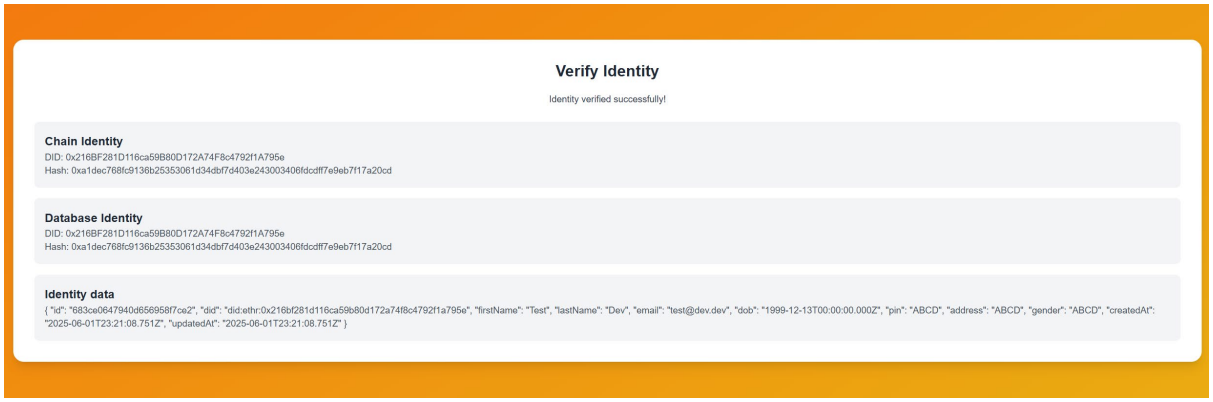


Figure 5.12: Orange IDP Identity Verification Page

The IDP also provides a dedicated verification interface, where users can compare the hash of their off-chain profile with the on-chain proof. If the hashes match, the user knows their identity has not been tampered with and is correctly anchored to their Ethereum address. This supports both personal trust and regulatory compliance for blockchain-backed identity systems.

Table 5.12: Orange IDP Collections List

Collection	Description
User	Stores identity attributes such as name, DOB, address and DID.
Nonce	Temporary collection used to store nonces for challenge-response verification.

The Orange IDP smart contract serves as the decentralized anchor for identity verification in the Orange Country node. Deployed on the Ethereum Sepolia testnet, it enables users to assert ownership over a Decentralized Identifier (DID) by binding it to a cryptographic hash of their off-chain identity data. The contract follows a privacy-preserving architecture: rather than storing personal attributes directly on-chain, it records only a keccak256 hash of the data. This ensures that while the integrity and origin of the identity can be verified, the actual content remains confidential and under the user's control. The contract also emits events (such as IdentityProofRegistered, Updated, and Deleted) to support transparency, traceability, and auditability without compromising privacy.

The interface of the smart contract exposes four primary functions: registerIdentityProof, updateIdentityProof, getIdentityProof, and removeIdentityProof. These endpoints allow wallet holders to register a new identity hash, update it as needed, retrieve identity metadata for a given address, or completely remove their entry. Access control is enforced implicitly through msg.sender, ensuring that only the address owner can register, update, or delete their own identity proof. By maintaining a one-to-one mapping between Ethereum addresses and identity records, the contract guarantees non-repudiation while enabling off-chain services (such as the Orange IDP backend) to verify identity claims during login or account recovery flows.

The design of this contract complements the hybrid nature of the Orange Country identity system. It integrates with an off-chain MongoDB-backed identity database, and its hashes are computed using the same structured data fields during registration and verification. This alignment ensures that the on-chain and off-chain records remain synchronized and verifiable. In future iterations, this mechanism could be extended to serve as a common identity registry across all nodes in the federation, enabling cross-border verification without centralized coordination, moving the architecture closer to a fully decentralized identity resolution protocol.

Table 5.13: Orange IDP Smart Contract Endpoints

Endpoint	Description
`getIdentityProof`	Retrieves the Decentralized Identifier (DID) and associated identity data hash for a given wallet address. This allows third parties to verify that a user has registered a valid identity proof.
`registerIdentityProof`	Registers a new identity proof by linking a wallet address to a DID and a hashed representation (keccak256) of the user's off-chain identity data. This function is permissionless but enforces uniqueness per address.
`updateIdentityProof`	Updates the hash of the identity data for a previously registered identity. The DID remains unchanged, while the new hash reflects the latest off-chain attributes.
`removeIdentityProof`	Deletes the identity proof associated with the caller's wallet address. This function allows users to revoke their identity claim and removes the corresponding entry from the smart contract storage.

Ultimately, the Orange Country IDP showcases how decentralized infrastructure, modern web technology, and federated authentication can be combined into a unified, scalable, and user-centric identity solution. In the next section, we will describe the user-facing pages, backend endpoints, and data model that power this system.

6. Future Work

While the current system demonstrates a functioning, privacy-preserving federated identity infrastructure, there remain multiple areas where the architecture could evolve further to achieve greater resilience, decentralization, and usability. This section outlines several planned improvements, which aim to address technical limitations and usability gaps in the prototype implementation.

6.1. Decentralized Identity Resolution and Duplication Prevention

The current identity discovery process relies on a centralized, trusted intermediary: the discovery gateway. While the gateway plays a crucial role in routing login requests and resolving identity ownership across nodes, it introduces a level of logical centralization. This not only makes it a potential single point of failure but also requires all participants to trust its behaviour and availability. In a truly decentralized system, such a component would ideally be eliminated or at least minimized in responsibility. To remove the need for centralized discovery logic, each country node could publish signed entries to a shared public or consortium blockchain. These entries would contain:

- A hashed identifier (e.g., keccak256(DID) or anonymized fingerprint),
- The node's metadata (e.g., public key or node ID),
- A digital signature proving authenticity and authorship.

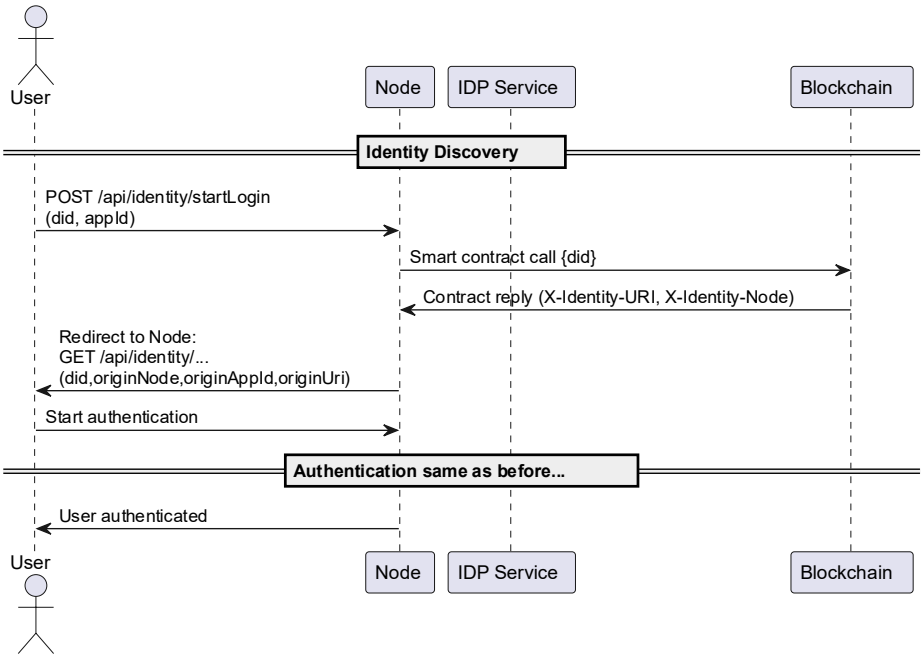


Figure 6.1: Authentication Flow with Smart Contract Discovery

By querying the blockchain, any node could discover which peer has previously registered a given identifier, without leaking sensitive user information. This on-chain registry would act as a tamper-resistant index of ownership claims, enabling secure, decentralized identity

resolution without point-to-point polling. Such smart contracts could be designed to reject conflicting submissions. For example, if a DID has already been registered by another node, a second claim would either be rejected or logged as a conflict requiring resolution. This serves as a global uniqueness constraint, similar to how DNS registries enforce non-duplication.

In decentralized identity systems, the risk of identity duplication arises when two or more nodes unknowingly register the same person under different records, possibly leading to fragmented digital identities. This is a particular concern in federated models without a strong global consistency layer. With blockchain-based publishing, identity duplication can be proactively detected and flagged. Because the hash of identity data (e.g., name, birthdate, etc.) is also published (as is already done in the Orange Country implementation), comparing these hashes across nodes allows for deduplication, even without sharing the full user data.

When a duplicate is detected, resolution policies can be applied:

- Temporal priority (first registration wins),
- Node reputation or legal hierarchy (e.g., eIDAS-certified issuers have precedence),
- User-initiated migration with cryptographic consent.

Beyond discovery, this decentralized ledger can serve a second role: enabling third parties (e.g., applications, auditors) to verify identity claims without needing real-time access to the origin node. By retrieving the latest signed identity proof and public key of the issuer from the chain, verifiers can independently validate whether a given identity is authentic and current. This turns identity proofs into verifiable credentials that are not only machine-readable but also self-validating, reducing the need for constant backend communication and increasing scalability.

Technically, this system could leverage Ethereum-compatible blockchains (e.g., Sepolia, Polygon, or L2 rollups), using:

- Smart contracts for storing identity hashes and metadata,
- EIP-712-style signatures to ensure forward compatibility,
- IPFS or similar decentralized storage for metadata (if extended attributes are needed),

This blockchain-backed registry would act as a global trust anchor, enabling secure interoperability among sovereign identity providers while maintaining decentralization, privacy, and verifiability.

6.2. Country nodes admin panel

To improve the maintainability, usability, and operational efficiency of the system, a dedicated administrative interface for managing country nodes is proposed as a key area for future development. While the current system relies on manual configuration, environment files, and direct database access to manage registered applications and scopes, this approach limits scalability and introduces risks in environments with multiple operators or federated governance.

A web-based admin panel would serve as a secure control surface for country node operators. This panel would enable role-based access control (RBAC) for administrators and allow real-time interaction with the node's core subsystems, including:

- Application Registry Management: Administrators could create, edit, or disable applications directly through the UI. The interface would allow setting allowed redirect URIs, assigning scope permissions, and viewing audit trails for app activity. Additionally, a search and filter feature would facilitate easier navigation of large registries.
- Scope Definition and Governance: While scopes are currently defined via code or database migration scripts, the admin panel could expose this logic as a dynamic, human-readable interface. Operators could create new scopes, attach documentation, and enforce cross-country compatibility standards.
- Federation Configuration: Each country node connects to a federation of peers (via the gateway or future blockchain-based mechanisms). The admin panel could allow authorized operators to view peer metadata, accept/reject new nodes, manage routing rules, and update local trust policies based on federation health or governance decisions.
- Monitoring and Logs: Admins would benefit from visibility into authentication flows, errors, and request traces. A dashboard summarizing recent user login requests, application activity, and proxy calls between countries would help detect misconfigurations or potential abuse.
- Security Management: Administrators would be able to rotate keys, configure CORS policies, manage public metadata (e.g., OpenID configuration), and control exposure of node endpoints based on trust levels.

In future implementations, this admin panel could be offered as a reusable component or service shared across deployments, allowing both centralized and decentralized governance depending on the federation model in use.

6.3. User migration flow

As decentralized systems mature, user mobility between jurisdictions becomes not only desirable but essential. In the current architecture, each user is intrinsically tied to a specific identity provider within a given country node. This reflects the structure of traditional identity systems, where issuers are geographically or organizationally bound. However, enabling users to migrate their identity across nodes (while retaining their unique identifier and minimizing data loss) is a critical step toward realizing the vision of user-centric, globally portable identities.

A future user migration flow would allow individuals to initiate a transfer of identity records from one country node to another, facilitating relocation, federation restructuring, or jurisdictional preference. The design of such a flow must balance three core goals: trust preservation, data integrity, and minimal user friction. I propose the following migration process:

1) Migration Request Initiation: The user authenticates to their current (origin) country node and initiates a migration request via a dedicated endpoint or user-facing interface. The request includes the target node, user consent, and any required legal or operational metadata.
2) Authorization and Consent: The origin node prompts the user to verify their identity and confirm the migration. It also creates a signed migration assertion, encoded as a JWT or verifiable credential, containing user claims, DID, and the node's cryptographic signature.
3) Data Packaging and Export: The origin node exports the user's identity data in a normalized, schema-compliant format, consistent with the application's OIDC-compatible internal representation. Sensitive fields (e.g., birthdate, address, PIN) are selectively included based on requested scopes and user approval.
4) Delivery to Target Node: The target country node receives the signed assertion and data bundle via a secure backend channel or through user-facilitated forwarding (e.g., download/upload). It verifies the origin signature and checks if the migration complies with local policies.
5) Conflict Resolution: If the user's identifier (e.g., DID) already exists in the target system, the user is prompted to merge accounts, override data, or choose an alias. Automated tools may also flag inconsistencies or duplication.
6) Registration and Confirmation: Once validated, the target node registers the user as a new local identity. Optionally, it publishes an updated DID association (e.g., via blockchain or internal registry) to inform the federation about the new location of the identity.
7) Revocation of Old Identity (Optional): The original node may revoke or deactivate the previous user entry, either immediately or after a defined grace period, ensuring there are no parallel representations unless explicitly allowed.

The introduction of a user migration flow offers significant benefits within a federated, decentralized identity system. First and foremost, it enhances interoperability by enabling individuals to carry their verified digital identities across national or organizational boundaries without the need for re-registration. This eliminates data redundancy, minimizes onboarding friction, and respects the continuity of user trust. Migration also contributes to system resilience, allowing identities to be transferred away from nodes that may become deprecated, compromised, or subject to shifting legal frameworks. Moreover, the ability to relocate a digital identity aligns with the principle of portability, empowering users to choose their preferred jurisdiction and regain control over where and how their identity is managed.

An effective migration mechanism also introduces the notion of trust delegation, where a verified identity and its associated trust are handed off from one authority to another using digitally signed assertions. This provides a formalized and cryptographically verifiable pathway for new nodes to accept identities from established participants, thereby scaling the federation securely and organically.

Looking ahead, several considerations will influence the design and success of such a feature. One important aspect is the integration of blockchain-based identity anchors, where each node's user registry could optionally publish or update identity location metadata on a distributed ledger. This would allow the network to resolve the authoritative source of a given identifier, even after migration. Additionally, federation-wide revocation registries could prevent users from unintentionally duplicating identities or bypassing local policies, thereby maintaining consistency and accountability.

Data protection and legal compliance will also be central. Migration flows must strictly follow data minimization principles, sharing only the attributes necessary for continued authentication or verification in the new domain. User consent logging, especially for sensitive information such as PINs or biometric references, must be transparent and verifiable to meet privacy regulations such as the GDPR and evolving requirements under eIDAS 2.0.

In conclusion, the user migration flow represents a key evolutionary step in decentralized identity infrastructure. By decoupling identity from a fixed issuer and enabling secure, policy-aware transfers between nodes, the system moves closer to true user sovereignty and global trust interoperability. This capacity not only supports technical flexibility but also reflects the social and regulatory reality of modern identity: dynamic, portable, and user centred.

7. Conclusions

THE system presented in this thesis offers a novel and pragmatic approach to decentralized identity management for web applications, combining the autonomy of localized identity providers with the interoperability of a federated network. By delegating authority to individual country nodes, each responsible for managing its own registry of applications and identity services, the architecture accommodates a broad range of technologies and trust models, from traditional OAuth2-based flows to blockchain-backed identity validation.

One of the defining features of the architecture is its dynamic identity discovery mechanism, which enables real-time resolution of a user's origin based on their decentralized identifier (DID). Unlike traditional systems such as eIDAS v1, where each node must explicitly peer with every other node to exchange user data, this solution centralizes discovery in a neutral gateway that operates without holding any user data. The gateway queries participating nodes in parallel and responds only with routing metadata, allowing the origin node to transparently redirect authentication to the appropriate peer. This model not only simplifies scaling and configuration but also preserves user privacy and enforces protocol uniformity across the federation.

The project includes two country implementations to illustrate the system's flexibility. Yellow Country uses a conventional Express.js-based OpenID Connect (OIDC) identity provider with support for username/password and WebAuthn passkey authentication, demonstrating how legacy systems can integrate into the federation. Orange Country, by contrast, uses a fully decentralized Ethereum-based login system that registers identity hashes on-chain and authenticates users through wallet signatures and smart contract queries. Despite their vastly different mechanisms, both implementations conform to the shared federation protocol, enabling cross-node login and shared authentication flow management.

Deployment is fully containerized and automated through cloud-init scripts, enabling rapid provisioning and consistent configuration across virtual machines. Internal network access and the use of secure, custom HTTP headers ensure that inter-node communication remains isolated and verifiable, even without cryptographic request signing. Future extensions, such as blockchain-based global identity indexing, node-level administrative dashboards, and secure user migration flows, are planned to increase transparency, accountability, and resilience.

In conclusion, this system represents a scalable and extensible framework for decentralized digital identity that bridges the gap between emerging decentralized identifiers and established authentication infrastructures. It demonstrates that a federation of autonomous identity providers can coexist and cooperate without sacrificing usability, privacy, or trust, and provides a blueprint for future-ready digital identity ecosystems beyond national borders.

Bibliography

[1] R. Allen and A. G. Lowe-Norris, Active Directory - Second Edition, O'Reilly, 2003.

[2] R. Kumar, "Enhancing API Security: A Comparative Analysis of OAuth 2.0, OpenID Connect, and SAML," *International Journal of Innovative Research in Engineering & Multidisciplinary Physical Sciences,* vol. 8, no. 3, 2020.

[3] European Parliament and Council of the European Union, "REGULATION (EU) No 910/2014 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 23 July 2014 on electronic identification and trust services for electronic transactions in the internal market and repealing Directive 1999/93/EC," *Official Journal of the European Union,* vol. L257, p. 73–114, 2014.

[4] European Parliament and Council of the European Union, "REGULATION (EU) 2024/1183 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework," *Official Journal of the European Union,* vol. L, April 2024.

[5] B. Arkills, LDAP Directories Explained: An Introduction and Analysis, Addison-Wesley Professional, 2003.

[6] D. Hardt; Internet Engineering Task Force (IETF), "The OAuth 2.0 Authorization Framework, RFC 6749," October 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749.html.

[7] European Comission, "eIDAS Regulation," 2024. [Online]. Available: https://digital-strategy.ec.europa.eu/en/policies/eidas-regulation. [Accessed 01 06 2025].

[8] S. Lips, N. Bharosa and D. Draheim, "eIDAS Implementation Challenges: The Case of Estonia and the Netherlands," in *Electronic Governance and Open Society: Challenges in Eurasia - 7th International Conference, EGOSE 2020*, 2020.

[9] H. Roßnagel, C. H. Schunck and E. S. Mödersheim, "eIDAS 2.0: Challenges, perspectives and proposals to avoid contradictions between eIDAS 2.0 and SSI," in *Open Identity Summit 2022, Lecture Notes in Informatics (LNI), Gesellschaft für Informatik*, Bonn, 2022.

[10] D. Berbecaru, A. Lioy and C. Cameroni, "Electronic Identification for Universities: Building Cross-Border Services Based on the eIDAS Infrastructure," *Information,* vol. 10, no. 6, 2019.

[11] Eduroam, "How does eduroam work?," [Online]. Available: https://eduroam.org/how/. [Accessed 01 Jun 2025].

[12] Berbecaru, Lioy, D. Gratiela, Cameroni, Antonio and Cesare, "Providing Login and Wi-Fi Access Services With the eIDAS Network: A Practical Approach," *IEEE Access,* vol. 8, pp. 126186-126200, 2020.

[13] S. Srinivas, D. Balfanz, E. Tiffany and A. Czeskis, “Universal 2nd Factor (U2F) Overview,” FIDO Alliance, 2017.

[14] D. George, “The Dawn of Passkeys: Evaluating a Passwordless Future,” *Partners Universal Innovative Research Publication,* vol. 02, no. 01, pp. 1-19, 2024.

[15] L. Lassak, E. Pan and B. Ur, “Why Aren't We Using Passkeys? Obstacles Companies Face Deploying FIDO2 Passwordless Authentication,” in *33rd USENIX Security Symposium*, Philadelphia, 2024.

[16] H. Haapaniemi, “Exploring Perceptions and Coping Behaviors Regarding Imposed Multifactor Authentication Usage: A University Case Study,” University of Jyväskylä, Jyväskylä, 2024.

[17] Apple, “WWDC25: What’s new in passkeys,” 10 Jun 2025. [Online]. Available: https://www.youtube.com/watch?v=mV68bUYVSL0. [Accessed 12 Jun 2025].

[18] Alizadeh, M. a. Andersson, K. a. Schelén and Olov, “Comparative Analysis of Decentralized Identity Approaches,” *IEEE Access,* vol. 10, pp. 92273-92283, 2020.

[19] Identity Foundation, “ION: A Decentralized Identifier (DID) Network,” 2024. [Online]. Available: https://identity.foundation/ion/. [Accessed 9 Jun 2025].

[20] F. Vogelsteller and T. Yasaka, “ERC-725: General data key/value store and execution - An interface for a smart contract based account with attachable data key/value store,” 2 Oct 2017. [Online]. Available: https://eips.ethereum.org/EIPS/eip-725. [Accessed 5 Jun 2025].

[21] W. Chang, G. Rocco, B. Millegan, N. Johnson and O. Terbu, “ERC-4361: Sign-In with Ethereum,” 11 Oct 2021. [Online]. Available: https://eips.ethereum.org/EIPS/eip-4361. [Accessed 5 Jun 2025].

[22] M. Holst and N. Johnson, “ERC-191: Signed Data Standard,” 20 Jan 2016. [Online]. Available: https://eips.ethereum.org/EIPS/eip-191. [Accessed 06 Jun 2025].

[23] R. Bloemen, L. Logvinov and J. Evans, “EIP-712: Typed structured data hashing and signing,” 12 Sep 2017. [Online]. Available: https://eips.ethereum.org/EIPS/eip-712. [Accessed 04 Jun 2025].

[24] Authority for Digitalisation of Romania, “Ghiseul.ro,” [Online]. Available: https://www.ghiseul.ro/ghiseul/public/login-eidas. [Accessed 16 Jun 2025].